



Pointwise Deep SABR Approximation for Swaption Markets

A thesis submitted in partial fulfilment of the requirements
for the degree in

Bachelor of Arts in Economics

submitted by

Vincent Klaer

21-614-458

Werderstr. 74

69120 Heidelberg

with

Thesis Advisor

Prof. Dr. Matthias R. Fengler

to

University of St. Gallen

November 2025

Abstract

This thesis investigates the use of a feedforward neural network as an approximation method for the implied volatility map of the SABR model, with a particular focus on swaption markets. The objective is to identify an approach that improves upon traditional approximations regarding both computational efficiency and accuracy. Using a two-step pointwise approach and a large synthetic training dataset, generated using Monte Carlo simulation, the neural network learns the mapping from SABR model parameters to implied volatilities. It is then evaluated on another synthetic dataset of swaptions, regarding its accuracy in generating implied volatilities from known SABR parameters, calibrating full volatility curves to recover the underlying SABR parameters, and producing option Greeks. Across nearly all evaluation metrics, the neural network approximation outperforms both the formula of Hagan et al. (2002) and the Zero Correlation Map of Antonov et al. (2013). Its principal shortcoming, however, lies in a higher number of arbitrage violations compared to other approximation methods.

Contents

1	Introduction	1
2	Literature Review	3
3	SABR Theory	5
3.1	Swaps and Swaptions	5
3.2	Black Model	6
3.3	SABR Model	8
3.4	Calibration to the Market	10
3.4.1	Hagan’s Formula	12
3.4.2	Zero Correlation Map	14
3.4.3	Monte Carlo Simulation	14
4	Neural Networks	16
4.1	Introduction to Neural Networks	16
4.2	One-Step vs. Two-Step Approach	17
4.3	Gridwise vs. Pointwise Approach	18
4.4	Hyperparameter Space	19
4.5	Hyperparameter Optimization	23
5	Methodology	25
5.1	Neural Network Setup	25
5.2	Train Data	26
5.3	Test Data	28
6	Discussion of Results	29
6.1	Neural Network Architecture	29
6.2	Monte Carlo Accuracy	30
6.3	Mapping Error	31
6.4	Calibration Error	36
6.5	Hedging Error	40
6.6	Arbitrage Violation	44

7 Conclusion	46
Bibliography	48
A Hyperparameter Optimization	53
B Monte Carlo vs. FDM Accuracy	55
C SABR Parameter Recovery	56
D Antonov et al. (2013) Approximation	57
List of Resources	60
Declaration of Authorship	61

List of Figures

3.1	Visualization of Changes in SABR Parameters ($f = 0.05$, $\tau = 1$, $\beta = 0.5$, $\alpha = 0.05 \rightarrow 0.1$, $\rho = -0.5 \rightarrow 0.5$, $\nu = 1 \rightarrow 2$)	8
4.1	Feedforward Neural Network Example (J. Liu et al., 2024, Figure 8)	17
6.1	Median Monte Carlo Error	30
6.2	Median Mapping Error: Hagan and Obłój	31
6.3	Median Mapping Error: Neural Network and ZC Map	32
6.4	Effect of τ and ν on Median Mapping Error: Neural Network, ZC Map, and Hagan	32
6.5	Maximum Mapping Error: Neural Network, ZC Map, and Hagan	34
6.6	Median Calibration Error: Neural Network, ZC Map, and Hagan	37
6.7	Maximum Calibration Error: Neural Network, ZC Map, and Hagan	38
6.8	Median Delta Error: Neural Network, ZC Map, and Hagan	41
6.9	Median Vega Error: Neural Network, ZC Map, and Hagan	42
6.10	Median Vanna Error: Neural Network, ZC Map, and Hagan	42
6.11	Median Volga Error: Neural Network, ZC Map, and Hagan	43
6.12	Butterfly Arbitrage Constraint Violations vs. Moneyness: Neural Network, ZC Map, and Hagan	45
A.1	Hyperparameter Optimization Results: Number of Hidden Layers & Mean Number of Nodes vs. MSE	53
A.2	Hyperparameter Optimization Results: Optimizer vs. MSE	54
A.3	Hyperparameter Optimization Results: Batch Size vs. MSE	54
C.1	Calibrated Parameter Error Distribution: Neural Network, ZC Map, and Hagan	56

List of Tables

6.1	Regression Results Comparing Neural Network, ZC Map, and Hagan Approximations	35
6.2	Average Calibration Time per Implied Volatility Curve	36
6.3	Median Absolute Parameter Errors: Neural Network, ZC Map, Hagan . . .	39
6.4	Arbitrage Violations Across Approximation Methods	44
A.1	Hyperparameter Ranges in Previous Literature (Table 1 of 2)	53
A.2	Hyperparameter Ranges in Previous Literature (Table 2 of 2)	53
B.1	Monte Carlo Simulation Bias and Variance Compared to Finite Difference Method (50 Trials)	55

List of Abbreviations

ATM	At the money
EI	Expected Improvement
FDM	Finite Difference Method
ITM	In the money
OTM	Out of the money
IV	Implied Volatility
MC	Monte Carlo
MSE	Mean Squared Error
SABR	Stochastic Alpha, Beta, Rho
TPE	Tree-Structured Parzen Estimator
USD	United States Dollar
ZC Map	Zero Correlation Map

1 Introduction

The accurate and quick modeling of implied volatility (IV) curves remains a crucial challenge for participants in interest rate derivatives markets. Among the various models available, the Stochastic Alpha, Beta, Rho (SABR) model, introduced by Hagan et al. (2002) has gained widespread adoption due to its ability to capture the implied volatility curve dynamics typically observed. A major factor in the model's adaptation has been its closed-form analytical approximation introduced by Hagan et al. (2002). While quick and computationally efficient, this solution suffers from deteriorating accuracy for long maturities and extreme strikes. Therefore, an improved version was derived by Oblój (2008), and more elaborate approximations, such as the Zero Correlation (ZC) Map by Antonov et al. (2013), followed. The ZC Map, while improving accuracy, suffers from high computational costs and, therefore, low calculation speed, which is of crucial importance in a trading environment. This leaves room for alternative approaches that can maintain the high accuracy while achieving the computational efficiency required.

Neural networks, in particular, offer considerable potential due to their universal function approximation properties and high computational speed. By training offline on large synthetic datasets of interest rate options generated through Monte Carlo simulation, a neural network can learn the nonlinear relationships embedded in the SABR model and subsequently reproduce them with minimal computational overhead.

This thesis investigates the application of a feedforward neural network to approximate the SABR model's mapping between parameters and implied volatilities, specifically within the context of interest rate derivatives. The results demonstrate that the neural network achieves computational efficiency close to Hagan's formula while outperforming both it and the Zero Correlation Map in reproducing the implied volatility mapping as well as in retrieving accurate parameters through market calibration. Moreover, the neural network closely captures the dynamic behavior of the SABR model relevant for computing SABR Greeks. The approximation does, however, exhibit certain difficulties in maintaining arbitrage-free implied volatility curves.

Firstly, a short literature review will serve to give an overview of previous research on approximating stochastic volatility models using neural networks, with a particular focus on studies related to the SABR model. The following section then provides a broad introduction to swaptions, explaining how implied volatilities are derived under the Black model and why this model fails due to non-constant volatility in real markets. This serves as a bridge to the SABR model, which is then discussed together with its most prominent approximations: Hagan’s formula, the ZC Map, and Monte Carlo simulations. The next chapter introduces neural networks and discusses how they can be trained to recreate the mapping of the SABR model. Here, important concepts from the literature review are revisited and further elaborated. Additionally, the key elements of a neural network architecture, known as hyperparameters, are briefly reviewed, along with the process of selecting their optimal combination through hyperparameter optimization. The thesis concludes with a description of the data and methods used, followed by an extensive analysis of the results, comparing the performance of the trained neural network against other approximation methods.

2 Literature Review

Feedforward neural networks have been around for a long time. However, their application in finance has only been of interest more recently, being spurred by papers such as those by Malliaris and Salchenberger (1993). This rise in interest was further perpetuated by a much broader availability of computational resources, allowing for the training of neural networks on large datasets.

The theoretical foundation for using neural networks for function approximation is provided by the Universal Approximation Theorem, originally proposed by Cybenko (1989). This result establishes that a feedforward neural network with a single hidden layer and a continuous sigmoidal activation function can approximate any continuous function on a bounded domain to arbitrary precision. This universal approximation capability makes neural networks a powerful tool for modeling the complex relationship between stochastic volatility model parameters and implied volatilities. Once trained, the network can, in theory, act as a surrogate model, efficiently replicating implied volatility curves.

Earlier research on applying neural networks to stochastic volatility models began with Hernandez (2017), who introduced the one-step or inverse approach. In this framework, a neural network directly learns the mapping from market-observed implied volatility curves to model parameters (Hernandez, 2017, pp. 1–2). Hernandez (2017) applied this concept to the Hull-White model, demonstrating its potential generality across stochastic volatility models. However, the approach was constrained by the limited availability of historical calibration data.

A significant methodological advance was brought forward by Bayer et al. (2019), who formalized the two-step approach. In this framework, as outlined by Bayer et al. (2019, pp. 9–11), a neural network is first trained offline to approximate the mapping from model parameters to implied volatilities. In a second step, calibration to the market-observed implied volatility curve is performed, and the learned map is inverted to recover the corresponding model parameters. Bayer et al. (2019) further distinguished between pointwise and gridwise learning, where in the former, the model estimates an implied

volatility for a single strike, while in the latter, it estimates multiple implied volatilities across a grid of strikes simultaneously (pp. 11–16). Bayer et al. (2019)’s experiments across various stochastic volatility models also showed that the SABR model could be accurately represented using only a shallow neural network, whereas rough volatility models required significantly deeper architectures (p. 16).

Building on this, Horvath et al. (2021) proposed a general two-step gridwise learning approach applied to the one-factor and rough Bergomi models. Compared to the Monte Carlo simulation, Horvath et al. (2021)’s neural network was more than 9,000 to 16,000 times faster while maintaining comparable accuracy (p. 21).

Focusing specifically on the SABR model, McGhee (2018) also implemented a two-step gridwise approach very early on. McGhee (2018) used a simple single-layer artificial neural network to learn the pricing map from an integration scheme for the SABR model, achieving fast computation with comparable accuracy to the original scheme (pp. 18–21). However, the work was limited to the lognormal SABR model ($\beta = 1$) and did not fully explore the calibration of volatility curves. Despite these limitations, the results demonstrated that even a simple neural architecture could substantially improve calibration speed and accuracy.

Most recently, Sala and Pravosud (2022) have addressed one of the main shortcomings of prior studies, the arbitrary selection of neural network hyperparameters. By adopting a two-step gridwise approach combined with automated hyperparameter optimization via the Tree-Structured Parzen Estimator, they offered a solution to this problem.

This thesis extends previous research by adapting the hyperparameter optimization framework while tailoring the neural network specifically for the calibration of volatility curves in interest rate derivatives markets. To reflect market conventions, where strike quotations are expressed in basis points relative to the forward rate, a pointwise rather than a gridwise approach is adopted. Additionally, the analysis goes beyond prior evaluation metrics by examining the dynamic behavior of the neural network approximation, which is crucial for accurately computing SABR Greeks. Lastly, violations of arbitrage constraints by the neural network approximation are detected and evaluated over the calibrated implied volatility curves.

3 SABR Theory

3.1 Swaps and Swaptions

Before turning to the details of the SABR model, it is essential to establish the financial context in which the model operates. The SABR model, while applicable to options in a variety of markets, is frequently used to model the dynamics of implied volatilities for interest rate options, also known as swaptions. Therefore, a concise overview of swaps and swaptions is provided below to clarify how these instruments are priced and why accurate modeling of their volatility curves is critical in practice.

As summarized by Luo (2016), in a standard interest rate swap, two counterparties agree to exchange cash flows based on different types of interest rates, typically one paying a fixed rate and the other a floating rate, over a defined time horizon from T_α to T_β (pp. 2–3). The contract specifies a notional principal amount on which interest payments are calculated, but the notional itself is not exchanged. Payments occur on predetermined dates $T_{\alpha+1}, \dots, T_\beta$. The floating leg is determined by the forward rate $f(T_{i-1}, T_i)$, set at T_{i-1} , and paid at T_i , with a year fraction $\tau_i = T_i - T_{i-1}$. The floating rate resets at each payment date, reflecting current market conditions. Given the yield curve, forward rates $f(T_{i-1}, T_i)$ can be computed for all future periods, allowing for a complete valuation of the swap. The fixed rate f_{fixed} is then chosen such that the present value of the fixed leg equals that of the floating leg at inception, ensuring that the swap initially has zero market value.

The present value of a payer swap (the contract paying the fixed rate and receiving the floating rate) at time t is given by:

$$P_{\text{swap}}(t) = L \sum_{i=\alpha+1}^{\beta} D(t, T_i) \tau_i (f(t; T_{i-1}, T_i) - f_{\text{fixed}}) \quad (3.1)$$

where $D(t, T_i)$ denotes the discount factor from time t to T_i , $f(t; T_{i-1}, T_i)$ represents the forward rate set at T_{i-1} for the accrual period $[T_{i-1}, T_i]$, and L refers to the notional principal.

Similarly, as described by Luo (2016), a swaption, specifically a European swaption, grants the holder the right, but not the obligation, to enter into an interest rate swap at a predetermined future date T_α (pp. 3–4). The two most common types are payer swaptions and receiver swaptions. The former gives the right to pay fixed and receive floating, while the latter does exactly the opposite. The present value of a European payer swaption is given by the following expression:

$$\text{PV}_{\text{payer}}(t) = L D(t, T_\alpha) \max \left(\sum_{i=\alpha+1}^{\beta} D(T_\alpha, T_i) \tau_i (f(T_\alpha; T_{i-1}, T_i) - K), 0 \right). \quad (3.2)$$

Here, $D(t, T_\alpha)$ is the discount factor from today to the exercise date, $D(T_\alpha, T_i)$ the discount factor at exercise to the i -th swap payment, $f(T_\alpha; T_{i-1}, T_i)$ the forward rate set at T_α for the period $[T_{i-1}, T_i]$, τ_i the year fraction, K the strike rate of the swaption, and L the notional principal.

While the formula presented provides the discounted intrinsic value of a European swaption, it does not capture its time value when evaluated at a time $t < T_\alpha$.

3.2 Black Model

To incorporate the time value of a swaption, the Black model¹, a simpler version of the well-known Black-Scholes-Merton model, can be adopted. Originally developed for pricing equity options on futures, the Black model has been adapted to interest rate options by treating the forward swap rate as the underlying asset. As discussed by Hagan et al. (2002), instead of modeling the entire term structure from T_α to T_β , this approach reduces the valuation to an option on a single forward swap rate f_t , which evolves according to a lognormal stochastic process (p. 3):

$$df_t = \sigma f_t dW_t \quad (3.3)$$

¹Many markets also quote implied volatilities under the Bachelier model, which works as well for negative interest rates.

Under these assumptions, the price of a European payer swaption can be expressed as:

$$P_{\text{payer}} = A L [f N(d_1) - K N(d_2)], \quad (3.4)$$

where

$$d_1 = \frac{\ln(\frac{f}{K}) + \frac{1}{2}\sigma^2\tau}{\sigma\sqrt{\tau}}, \quad d_2 = d_1 - \sigma\sqrt{\tau} \quad (3.5)$$

Here, L is the notional principal, A is the present value of the fixed-leg unit cash flows of the underlying swap, f is the forward swap rate at time 0, K is the strike rate (the fixed rate agreed upon if the swaption is exercised), $N(\cdot)$ is the cumulative standard normal distribution function, and σ is the implied volatility of the swaption, with maturity $\tau = T_\alpha - T_0$.

As explained by Hagan et al. (2002), all parameters in Black's formula are directly observable except for σ (pp. 3–5). In practice, for every combination of expiry (T_α) and tenor ($T_\beta - T_\alpha$), market participants can plot points of σ against the forward strikes, forming the implied volatility curve.

In theory, Black's formula assumes a constant volatility σ for all forward strikes. However, in reality, the implied volatilities vary systematically with forward strikes, producing the implied volatility curves observed in markets. Allowing σ to vary across strikes effectively means using a different model for every option, which undermines consistency. Because the Black model can only retrieve implied volatilities σ from swaptions with quoted prices, practitioners must find a way to obtain implied volatilities for unquoted strikes. This highlights the need for a consistent model. The issue becomes even more pronounced in the pricing of exotic options, which often depends on a continuum of strikes.

Furthermore, there are implications for the hedging of options. An example is the delta Greek, which measures an option price sensitivity to changes in the underlying forward rate. Under the Black model, this is computed using the implied volatility associated with each strike. As a result, deltas derived at different volatilities are not directly comparable. As Hagan et al. (2002) note, even a portfolio appearing delta-neutral through offsetting high and low strike option positions may retain residual exposure, since each delta is computed under a different implied volatility (p. 4). The vega Greek, which measures sensitivity to changes in implied volatility, is similarly inconsistent across strikes when each is associated with a distinct implied volatility. Together, these limitations underscore the need for a consistent model.

3.3 SABR Model

Building on the last section, a model that explicitly accounts for the random nature of volatility is required. This is where stochastic volatility models come into play, as they treat the volatility of the underlying asset as a random process that evolves over time, rather than being constant. Among these models, the Stochastic Alpha, Beta, Rho model, originally proposed by Hagan et al. (2002), has become one of the most widely used approaches for modeling implied volatility curves, especially in interest rate derivative markets. The SABR model introduces a two-factor stochastic framework where both the forward price and its volatility evolve according to the following stochastic differential equations (Hagan et al., 2002, p. 8):

$$\begin{aligned} df_t &= \alpha f_t^\beta dW_1, & f_0 &= f, \\ d\alpha_t &= \nu \alpha_t dW_2, & \alpha_0 &= \alpha, \end{aligned} \quad (3.6)$$

where f is the initial forward rate, α is the initial volatility, and the two processes are correlated by:

$$dW_1 dW_2 = \rho dt. \quad (3.7)$$

Additionally, the model parameters must satisfy:

$$\alpha, \nu > 0, \quad -1 \leq \rho \leq 1, \quad 0 \leq \beta \leq 1. \quad (3.8)$$

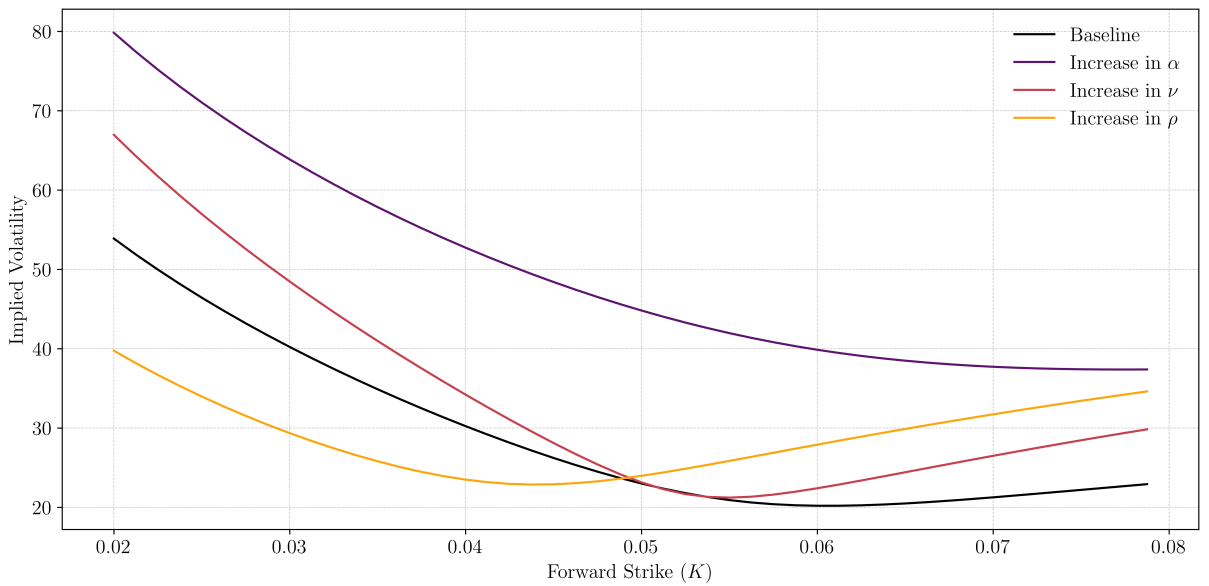


Figure 3.1: Visualization of Changes in SABR Parameters ($f = 0.05$, $\tau = 1$, $\beta = 0.5$, $\alpha = 0.05 \rightarrow 0.1$, $\rho = -0.5 \rightarrow 0.5$, $\nu = 1 \rightarrow 2$)

As outlined by Hagan et al. (2002), the SABR model is parameterized by the four key parameters α , β , ρ , and ν , each of which plays a distinct role in the stochastic process and in shaping the implied volatility curve (pp. 17–18). α represents the initial level of volatility of the SABR process, influencing the height of the volatility curve. However, it is important to note that a change in α does not correspond directly to a uniform shift in market-observed volatility across strikes. Next, the volatility of volatility, ν , controls the curvature. Higher values of ν imply a steeper curve. The parameter ρ determines the skew of the volatility curve, meaning the asymmetry in implied volatility between out of the money (OTM) and in the money (ITM) strikes. As ρ increases, the curve tilts, producing higher implied volatilities for higher strikes and vice versa. However, while the effects of changes in each parameter appear straightforward, parameters can interact in complex ways, causing different combinations to produce similar implied volatility curves.

Lastly, it is important to discuss the parameter β in the SABR model. Hagan et al. (2002) further elaborate on the importance of this parameter and detail the most common choices (pp. 11–12):

- $\beta = 1$: lognormal SABR model,
- $\beta = 0$: normal SABR model,
- $\beta = 0.5$: Cox-Ingersoll-Ross square-root process SABR model.

β can be determined either from historical observations of the so-called backbone or chosen based on aesthetic considerations. The backbone refers to the curve traced by the at the money (ATM) implied volatility $\sigma_{\text{ATM}}(f)$ as a function of the forward price f . One way to estimate β empirically is through a log-log plot of historical f - σ_{ATM} pairs (the procedure is further elaborated in Hagan et al. (2002, p. 12)). However, this method tends to produce noisy results. Since both β and ρ influence the shape of the implied volatility curve in a similar way, market curves can generally be fit equally well with different β values. Additionally, it is worth noting that fitting β to each curve directly would amount to fitting the noise rather than capturing a stable model structure.

As outlined by Hagan et al. (2002), β is typically selected to reflect the qualitative characteristics of the underlying market (p. 12). For example, $\beta = 1$ is common in foreign exchange markets, where lognormal dynamics are often seen as more natural. In contrast, $\beta = 0$ is preferred in low-rate environments, like the Yen interest rate market. A middle-ground choice of $\beta = 0.5$ is frequently adopted by U.S. interest rate desks.

Accordingly, $\beta = 0.5$ is used in this thesis, for which Rebonato et al. (2012) provide further strong support (pp. 43–44). Using a dataset of USD swaption prices between 2004 and 2007, the authors show that under $\beta = 0$, the parameters ρ and ν exhibit significantly greater day-to-day variation compared to when $\beta = 0.5$ is used. This highlights the practical stability of $\beta = 0.5$ and supports its adoption on empirical grounds. Beyond data-driven justification, Rebonato et al. (2012) also suggest an explanation based on a game theory approach: once a modeling standard like SABR with $\beta = 0.5$ gains traction in the market, traders are naturally incentivized to adopt it. This shared convention simplifies communication, aligns risk practices, and promotes consistency across market participants, creating a self-reinforcing cycle of adoption, even if the choice is not strictly grounded in theory.

3.4 Calibration to the Market

Now the question remains how to obtain the SABR parameters consistent with a given implied volatility curve. This is done in the calibration process, which aligns the model’s output implied volatilities with market-observed implied volatilities.

As outlined by Sala and Pravosud (2022), let the vector of model parameters to be calibrated be denoted by $\theta = (\alpha, \nu, \rho)$, and let $\gamma = (K, \tau, f)$ represent the observable, option-specific inputs, namely strike, maturity, and forward rate (p. 18). The market provides a matrix of implied volatilities $IV_t^{\text{mkt}}(K_i, \tau_j)$, where each element corresponds to a particular combination of strike K_i and maturity τ_j . The calibration is performed independently for each τ , where the objective is to find the parameter vector θ that minimizes the mean squared error between the model’s implied volatilities and the observed market volatilities across each implied volatility curve. Since the analytical form of $IV_t(\theta, \gamma)$ in the SABR model is unavailable, it is replaced with an approximation $\widetilde{IV}_t(\theta, \gamma)$. The calibration problem can then be formulated as:

$$\hat{\theta} = \arg \min_{\theta \in \mathbb{R}^p} \sum_{i=1}^k \sum_{j=1}^l q_{\gamma_{ij}} \left[\widetilde{IV}_t(\theta, \gamma_{ij}) - IV_t^{\text{mkt}}(K_i, \tau_j) \right]^2, \quad (3.9)$$

which can be solved using an optimization algorithm.

$q_{\gamma_{ij}}$ denotes a weighting factor that adjusts the relative importance of each implied volatility quote in the calibration, for example, to account for liquidity or bid–ask spreads. The process is repeated for each maturity T_j , resulting in a set of calibrated parameter vectors $\hat{\theta}_j$, each corresponding to a different maturity slice (Bayer et al., 2019, p. 5).

Unlike Black's model, the SABR model lacks an exact closed-form solution, connecting θ and γ to implied volatility. Therefore, an approximation for mapping $\widetilde{IV}_t(\theta, \gamma)$ must be established. There are multiple such approximations, whose primary goal is to accurately reproduce the true SABR mapping. Beyond accuracy, approximations are guided mainly by two practical considerations.

Firstly, swaption pricing and Greek calculation must be performed extremely quickly, as traders often need to evaluate thousands of swaptions over the course of a single trading day. In addition, model calibration to the observed volatility curve demands high computational efficiency since the calibration procedure is iterative.

Secondly, a fundamental requirement for any option pricing model is the absence of arbitrage. The following arbitrage constraints are outlined by Itkin (2019, p. 6). Payer swaptions are denoted with $P_{\text{payer}}(\tau, K)$, where τ is the maturity and K is the forward strike. Given forward strikes $K_1 < K_2 < K_3$, payer swaptions must exhibit the following properties:

Positivity condition

$$(I) \quad P_{\text{payer}}(\tau, K) > 0 \tag{3.10}$$

Vertical spread condition

$$(II) \quad P_{\text{payer}}(\tau, K_1) > P_{\text{payer}}(\tau, K_2) \tag{3.11}$$

Butterfly spread condition

$$(III) \quad (K_3 - K_2)P_{\text{payer}}(\tau, K_1) - (K_3 - K_1)P_{\text{payer}}(\tau, K_2) + (K_2 - K_1)P_{\text{payer}}(\tau, K_3) > 0 \tag{3.12}$$

Since violations of these constraints can be easily exploited, it is important for SABR approximations not to violate them.

3.4.1 Hagan's Formula

Hagan et al. (2002) proposed one of the most widely used approximation formulas for the SABR model, which is easy to implement and computationally efficient (p. 9). The authors use the singular perturbation expansion to the second order to obtain the implied volatilities of European options:

$$\widetilde{IV}_H = I_H^0 (1 + I_H^1 \tau), \quad (3.13)$$

$$I_H^0 = \frac{\alpha}{p \left(1 + \frac{(1-\beta)^2 q^2}{24} + \frac{(1-\beta)^4 q^4}{1920} \right)} \cdot \frac{z}{\lambda}, \quad (3.14)$$

$$I_H^1 = \frac{(1-\beta)^2 \alpha^2}{24 f^{2-2\beta}} + \frac{\rho \beta \nu \alpha}{4 f^{1-\beta}} + \frac{2-3\rho^2}{24} \nu^2, \quad (3.15)$$

$$p = (fK)^{\frac{1}{2}(1-\beta)}, \quad q = \ln\left(\frac{f}{K}\right), \quad z = \frac{\nu p q}{\alpha}, \quad \lambda = \ln\left(\frac{\sqrt{1-2\rho z + z^2} + z - \rho}{1-\rho}\right). \quad (3.16)$$

When $K \rightarrow f$, I_H^0 reduces to

$$I_H^0 = \frac{\alpha}{f^{1-\beta}}. \quad (3.17)$$

While the approximation by Hagan et al. (2002) has become popular for modeling implied volatility curves, it comes with several issues, as elaborated by Obłój (2008, pp. 2–3). Firstly, from a theoretical perspective, as $\beta \rightarrow 1$, I_H^0 fails to converge to its known value. Furthermore, it has been observed that in regions of small strikes and large maturities, the approximated implied volatility can be very far from the actual volatility. In addition, in those areas, the formula can assign negative prices to options that possess positive payoffs.

Besides a critique, Obłój (2008) also defines a refinement of Hagan's approach (pp. 2–3). Obłój (2008) compares the Hagan formula with an alternative result by Berestycki et al. (2004), showing that Berestycki's formula, for $\beta < 1$, better captures the correct option prices at small strikes and large maturities while addressing the convergence issue.

Following Xiong (2011, pp. 2–6), in equation (B.57c) of Hagan et al. (2002),

$$\zeta = \frac{\nu}{\alpha} \int_K^f \frac{dF}{F^\beta} = \frac{\nu}{\alpha} \eta \quad (3.18)$$

η is replaced with its approximation, thereby truncating higher-order terms:

$$\zeta = \frac{\nu}{\alpha} \eta = \frac{\nu pq}{\alpha} \left(1 + \frac{(1-\beta)^2 q^2}{24} + \frac{(1-\beta)^4 q^4}{1920} + \dots \right) \quad (3.19)$$

Oblój (2008)'s correction instead uses $\zeta = \frac{\nu}{\alpha} \eta$, where

$$\eta = \int_K^f \frac{dF}{F^\beta} = \begin{cases} \frac{f^{1-\beta} - K^{1-\beta}}{1-\beta}, & \beta < 1, \\ \ln\left(\frac{f}{K}\right), & \beta = 1, \end{cases} \quad (3.20)$$

modifying the leading-order term I_H^0 in the Hagan approximation:

$$I_{Ob}^0 = \frac{\nu q}{\lambda}, \quad (3.21)$$

$$p = (fK)^{\frac{1}{2}(1-\beta)}, \quad q = \ln\left(\frac{f}{K}\right), \quad \zeta = \frac{\nu}{\alpha} \eta, \quad \lambda = \ln\left(\frac{\sqrt{1-2\rho\zeta+\zeta^2} + \zeta - \rho}{1-\rho}\right). \quad (3.22)$$

For $K \rightarrow f$, I_{Ob}^0 also reduces to

$$I_{Ob}^0 = \frac{\alpha}{f^{1-\beta}}. \quad (3.23)$$

While this modification improves the theoretical consistency of the approximation, it does not fully eliminate all issues. As pointed out by Karlsmark (2013, p. 17), even the corrected formula tends to produce a too steep implied volatility curve, similar to that of the original Hagan et al. (2002) formula. Additionally, according to Choi and Wu (2021), it can also potentially give rise to butterfly arbitrage opportunities in the low strike region (p. 2). Under the analytic approximations, this occurs because an absorbing boundary condition is not actually imposed at the origin.

3.4.2 Zero Correlation Map

The Zero Correlation Map, introduced by Antonov et al. (2013), offers an improvement over Hagan’s and Obłój’s SABR approximation formula (pp. 2–4). Antonov et al. (2013) derive an exact pricing formula for the zero-correlation case ($\rho = 0$) with $0 < \beta < 1$. Using this solution, Antonov et al. (2013) introduce a mapping procedure based on heat-kernel theory that adjusts the SABR parameters to account for non-zero correlation. This approach uses the zero-correlation model as a mimicking model to approximate the solution under general correlation. This reparameterization, known as the ZC Map, preserves the small-time asymptotics of the full SABR model with non-zero correlation while allowing the reuse of the zero-correlation formulas (see Appendix D for the full option pricing formulas). Unlike the Hagan formula, which provides a direct expression for implied volatility, the Antonov et al. (2013) method first computes the option price. The implied volatility is then retrieved by inverting the Black formula.

The ZC Map has been shown to outperform other similar approaches based on heat-kernel expansions, such as those by Henry-Labordère (2008) and Paulot (2015), which rely on the Black-Scholes model as the mimicking model (Antonov et al., 2019, pp. 97–99). Additionally, according to Antonov et al. (2013), the approach should be nearly arbitrage-free for all practical purposes (p. 6).

Lastly, one important drawback of the ZC Map is the high computational burden imposed by having to solve two-dimensional integrals. A highly accurate approximation can be derived that reduces the problem to one-dimensional integrals (see Equations D.14–D.15), significantly alleviating the computational burden. However, even this simplified approach remains relatively computationally expensive compared to alternative methods. As noted earlier, this poses challenges for practical application, where implied volatilities must be calculated rapidly.

3.4.3 Monte Carlo Simulation

Monte Carlo (MC) simulation can be a very accurate approximation for mapping parameters to implied volatilities in the SABR model. The method consists of simulating a large number of sample paths of the SABR stochastic differential equations up to the option expiry. Averaging these payoffs and discounting them back to the present yields an estimate of the option prices, from which the implied volatility at each strike can be obtained by back-solving the Black formula.

To approximate the sample paths in this paper, a simple Euler discretization scheme is used:

$$\begin{aligned}\hat{F}_{t+1} &= \hat{F}_t + \hat{\alpha}_t \hat{F}_t^\beta \Delta W_{1,t}, & \hat{F}_0 &= f, \\ \hat{\alpha}_{t+1} &= \hat{\alpha}_t + \nu \hat{\alpha}_t \Delta W_{2,t}, & \hat{\alpha}_0 &= \alpha.\end{aligned}\tag{3.24}$$

For each time step $\Delta = t_{n+1} - t_n$, the correlated increments are:

$$\begin{aligned}\Delta W_{1,t} &= \sqrt{\Delta} Z_{1,t}, \\ \Delta W_{2,t} &= \sqrt{\Delta} \left(\rho Z_{1,t} + \sqrt{1 - \rho^2} Z_{2,t} \right).\end{aligned}\tag{3.25}$$

where $Z_{1,t}$ and $Z_{2,t}$ are independent standard normal random variables.

As the number of time steps and simulation paths increases, the accuracy of the approximation improves. The SABR process has a non-zero probability of hitting zero for $0 < \beta < 1$ and the absorbing boundary condition holds naturally for $\frac{1}{2} \leq \beta < 1$, but should be explicitly imposed for $0 < \beta < \frac{1}{2}$ for the price process to be a martingale and arbitrage-free (Choi & Wu, 2021, pp. 2–4). For the Euler discretization, in contrast, an absorbing boundary at the origin needs to be imposed explicitly for $0 < \beta < 1$ (Choi et al., 2025, p. 8). Due to the low order of convergence, the method requires very fine time steps to minimize the discretization error, leading to high computational costs.

Alternative methods, such as piecewise semi-exact simulation by Cai et al. (2017) or low-bias schemes by Chen et al. (2012), have been proposed in order to achieve the same accuracy using fewer steps. Cai et al. (2017) also propose exact simulation for the SABR model in two special but practically interesting cases, i.e., when the elasticity $\beta = 1$, or when $\beta < 1$ and the price and volatility processes are instantaneously uncorrelated. While the semi-exact simulation and low-bias schemes do improve bias and variance for the same step size, when comparing accuracy at an equivalent computational cost, they do not necessarily outperform the Euler discretization scheme in terms of accuracy (Cai et al., 2017, p. 946). Moreover, their implementation is considerably more complex.

4 Neural Networks

4.1 Introduction to Neural Networks

To overcome the limitations inherent in the previously presented approximation methods, recent research has increasingly turned to feedforward neural networks. As outlined by S. Liu et al. (2019), a feedforward neural network is composed of three main components: neurons, layers, and the overall network structure that connects them (p. 7). Each neuron performs three basic operations: it computes a weighted sum of its inputs, adds a bias term, and applies an activation function to produce an output. The activation function determines whether and to what extent a neuron is activated. By stacking neurons into multiple layers, the network forms a hierarchy of nonlinear transformations, enabling it to model complex input-output mappings.

The fundamental feedforward architecture can be expressed as a composite function:

$$F(x \mid \omega) = f^{(L)}\left(\dots f^{(2)}(f^{(1)}(x; \omega^{(1)}); \omega^{(2)}); \dots \omega^{(L)}\right), \quad (4.1)$$

where each layer $f^{(i)}$ is parameterized by $\omega^{(i)} = (W^i, b^i)$, consisting of a weight matrix W^i and a bias vector b^i . Each individual layer $f^{(i)}$ of the neural network takes the form:

$$f^{(i)}(x; \omega^{(i)}) = \phi^{(i)}\left(W^{(i)}x + b^{(i)}\right), \quad (4.2)$$

where $\phi^{(i)}$ is the activation function.

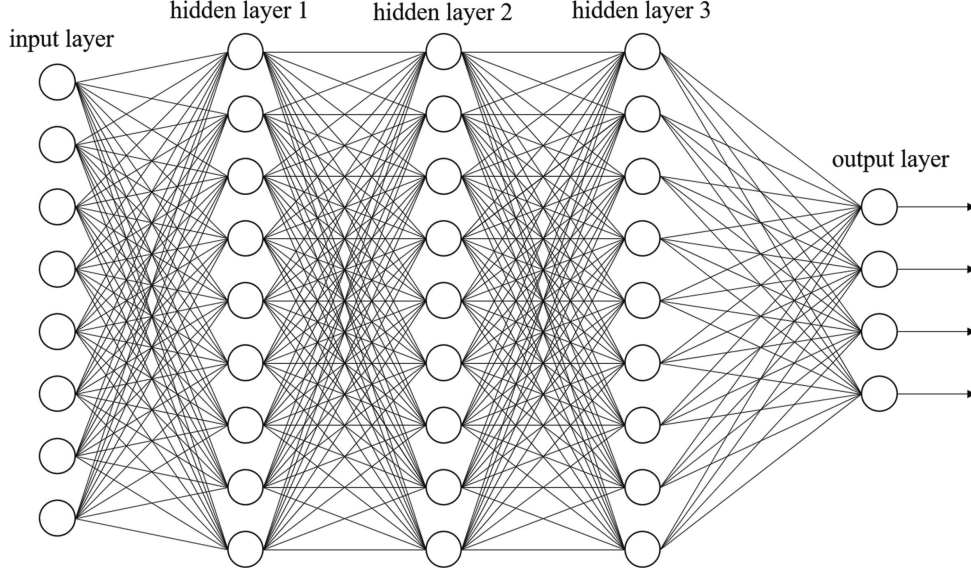


Figure 4.1: Feedforward Neural Network Example (J. Liu et al., 2024, Figure 8)

To learn the optimal weights, the neural network must be trained on a representative dataset that captures the relationships it is intended to approximate. The objective is to find the optimal weight configuration $\hat{\omega}$ that minimizes the mean squared error (MSE) between the neural network's predicted outputs and the corresponding target values from the dataset.

4.2 One-Step vs. Two-Step Approach

As elaborated by Bayer et al. (2019), two main training frameworks have emerged in the context of approximating the mapping from stochastic volatility models, such as the SABR model (p. 6). The one-step approach, originally pioneered by Hernandez (2017), uses a neural network to learn a direct mapping from market quotes to model parameters. The idea is to train a feedforward neural network that takes market-implied volatilities as input and returns the corresponding SABR parameters as output. The two-step approach follows a slightly more involved process. In the first step, a neural network is trained to approximate the implied volatility mapping, using SABR and market parameters as input and implied volatilities as output. In the second step, the neural network's output is calibrated to market data, allowing the SABR parameters to be retrieved by inverting the learned mapping.

This approach offers several advantages over the one-step method. Firstly, a pre-trained neural network can be differentiated with respect to the model parameters to obtain the SABR Greeks. Secondly, it provides greater interpretability, as the output of the

trained neural network can easily be compared to other approximation methods, such as Hagan’s formula or the ZC Map. This goes hand in hand with the fact that existing risk management model libraries can be retained with only minimal adjustments. Lastly, the training process becomes more robust against generalization errors on unseen data, as the trained neural network is independent of specific market biases and remains unaffected by shifts in market conditions.

Given all these advantages, this thesis addresses the calibration of the implied volatility curve for swaptions using a two-step approach.

4.3 Gridwise vs. Pointwise Approach

Another important choice involves how to structure the input and output of the neural network. Bayer et al. (2019) discuss both the pointwise and gridwise approaches (p. 6). In the pointwise approach, the neural network is trained to approximate the implied volatility for one specific option. The network thus produces only a single output, the implied volatility. This configuration allows the neural network to predict implied volatilities flexibly across any range of strikes, as long as it is trained on them. In contrast, the gridwise approach fixes a predefined vector of strikes. The neural network then learns to map from SABR parameters directly to an implied volatility curve defined over that fixed grid. Rather than predicting a single implied volatility, the network outputs a full vector of implied volatilities corresponding to the grid.

Bayer et al. (2019) evaluate both methods, noting that the main advantage of the gridwise approach lies in its computational efficiency, as it eliminates the need to treat strikes as dynamic inputs (pp. 14–16). A single evaluation of the neural network is sufficient to produce implied volatilities across a wide range of strikes. Moreover, this structure allows for the use of smaller and more compact networks that can still achieve high accuracy. Another benefit is the reduction in the variance of the training data, since only the model parameters are varied and the strike grid is fixed. This setup improves learning stability and efficiency. The pointwise approach, by contrast, offers more flexibility. It allows for targeting specific parts of different volatility curves, for example, by focusing on regions of the market where instruments are most liquid or most frequently traded. The pointwise method also addresses a notable drawback of the gridwise method, which is the lack of automatic interpolation. Because the network only outputs implied volatilities at fixed grid points, external interpolation is required to estimate values at untrained strikes. Bayer et al. (2019) argue that the implied volatility surface is typically smooth and well-

behaved as a function of maturity and strike (τ, K) , given fixed SABR parameters θ , easing interpolation. However, this assumption may not hold in regions where the volatility curve is sharply curved, which can result in interpolation errors and, therefore, reduced accuracy under the gridwise method.

Two key considerations motivate the use of a pointwise approach in this thesis. Firstly, there are times in swaption markets when the implied volatility curves show large kinks. Such irregularities may arise when model parameters assume unusually high values, for example, during the period between 2008 and 2015 in the U.S., when interest option prices still reflected a binding zero lower bound (Roppel et al., 2023, pp. 13–15). Secondly, swaption strikes are typically quoted in relative basis points relative to the forward rate f . Consequently, the absolute strike value K depends directly on the prevailing forward rate, which shifts over time. These characteristics make it impossible to define a fixed strike grid during training, as required in the gridwise setup. The pointwise approach, by contrast, accommodates these fluctuations naturally by treating strike and maturity as input variables, allowing flexible and accurate modeling across varying market conditions.

Using the two-step and pointwise approach, the training dataset connects vectors of SABR parameters $\theta_i = (\alpha_i, \beta_i, \rho_i, \nu_i)$ and market inputs $\gamma_i = (\tau_i, \frac{K_i}{f_i})$, where the forward rate f_i and strike K_i are combined into a single input to represent moneyness. Each pair (θ_i, γ_i) corresponds to an implied volatility $\widetilde{IV}^{\text{MC}}$, computed via Monte Carlo simulation, which serves as the training target. The neural network then learns to replicate this mapping by outputting $\widetilde{IV}^{\text{NN}}$. The training objective is to determine the optimal weights and biases $\hat{\omega}$ that minimize the mean squared error between the network’s predicted implied volatilities and the Monte Carlo-generated targets across all samples in the training set:

$$\hat{\omega} = \arg \min_{\omega \in \mathbb{R}} \frac{1}{N} \sum_{i=1}^N \left(\widetilde{IV}^{\text{NN}}(\theta_i, \gamma_i; \omega) - \widetilde{IV}^{\text{MC}}(\theta_i, \gamma_i) \right)^2. \quad (4.3)$$

4.4 Hyperparameter Space

Another important step is to decide on the neural network’s hyperparameters, its so-called topology. This requires careful consideration, as these architectural choices strongly influence the network’s ability to approximate the SABR mapping accurately. To this end, it is essential to define the hyperparameter choices from which the optimal configuration can be selected.

Nodes and Hidden Layers

The most critical aspect of a neural network’s architecture is the number of nodes and hidden layers. Previous research has shown that increasing network depth substantially enhances its expressive power. For instance, Telgarsky (2016) demonstrates that certain functions can be represented by deep networks with approximately k^3 layers and constant width but cannot be reproduced by shallower networks with only k layers unless those shallow networks become exponentially wide, requiring at least 2^k nodes (p. 1). Building on this, Lu et al. (2017) show that narrow networks struggle to approximate the representational capacity of wider ones unless they dramatically increase the number of nodes (p. 8).

Regarding the specific application of neural networks in determining the mapping of stochastic volatility models, width and depth often depend heavily on the type of model. Bayer et al. (2019) state that while for the calibration of classical stochastic volatility models, such as SABR or Heston, a shallow network architecture is sufficient, rough volatility models require a deeper network design (p. 4). According to S. Liu et al. (2019), increasing the number of nodes or layers can improve approximation accuracy, but this comes at the cost of increased computational complexity during training and evaluation (p. 9).

Optimizer

To train the neural network, the loss function (see Equation 4.3) must be minimized through an iterative process. According to Kingma and Ba (2015), optimizers can be broadly divided into gradient-based and gradient-free methods (p. 5). The focus of this thesis lies solely on gradient-based optimizers, which update the model parameters by computing the gradient of the loss function with respect to the weights and biases ω , and then adjusting them in the direction of steepest descent to minimize the loss. The size of each update step is determined by the learning rate η . This process is repeated iteratively over mini-batches of training data, gradually refining ω .

Among gradient-based methods, the Adam optimizer introduced by Kingma and Ba (2015) is the most widely used in the literature for approximating the mapping of stochastic volatility models. Adam computes individual adaptive learning rates for each parameter based on the first and second moments of the gradients (p. 1). This adaptive adjustment helps smooth the gradient descent trajectory.

Another widely used gradient-based optimization algorithm is RMSProp. RMSProp adjusts the learning rate for each parameter by scaling the gradient using the square root of an exponentially weighted moving average of past squared gradients (PyTorch 2.9 Documentation, n.d.-b). According to Kingma and Ba (2015), a key distinction is that RMSProp lacks a bias-correction mechanism, which may lead to overly large step sizes and, therefore, instability during training (p. 5).

Activation Function

The activation function transforms the input signal of each neuron into an output signal that is then passed to the next layer. Activation functions introduce non-linearity into neural networks, allowing them to capture complex, non-linear relationships between inputs and outputs. Three activation functions are most commonly used in prior research on approximating stochastic volatility models with deep learning techniques: ReLU, ELU, and Softplus.

However, as elaborated by Itkin (2019), both ReLU and ELU exhibit certain drawbacks (pp. 4–5). ELU can produce negative outputs, which is problematic when used in the final layer, since implied volatilities must remain strictly positive. Furthermore, its first derivative is generally not continuous. Although under specific conditions the first derivative of ELU can be made continuous, its second derivative remains discontinuous at zero, which is an undesirable property when computing higher-order Greeks. ReLU, on the other hand, is not continuously differentiable even in the first order, potentially causing jumps in first order Greeks and inactive regions with zero gradients when the domain is negative.

Given these limitations, the Softplus function emerges as the most suitable activation function for the neural network, as it ensures positivity, smoothness, and continuous differentiability of the first and second order. Softplus is defined as (PyTorch 2.9 Documentation, n.d.-c):

$$\phi(z) = \frac{1}{\beta} \ln(1 + e^{\beta z}), \quad (4.4)$$

where $\beta = 1$ in this thesis.

Mini-Batch Size and Number of Training Epochs

As explained by Goodfellow et al. (2016), an epoch represents one complete pass of the entire training dataset through the model (pp. 274–276). In practice, the dataset is divided into mini-batches, which are processed sequentially within each epoch. The mini-batch size refers to the number of training samples used to compute one update of the model’s weights and biases ω during the gradient-based optimization process. A smaller mini-batch size results in more frequent updates but generally requires more iterations for convergence. While larger mini-batch sizes may appear more efficient, smaller mini-batches often improve generalization, whereas excessively large mini-batches can cause the optimizer to converge to local minima, leading to overfitting and weakening the model’s ability to generalize.

Training over multiple epochs allows the model to iteratively refine its parameters and improve overall performance. Selecting the number of training epochs is important. Too few epochs may lead to underfitting, while too many can result in overfitting. To mitigate overfitting, Goodfellow et al. (2016) recommend the use of early stopping, a technique that halts training once the model shows no meaningful improvement on the validation set after a predetermined number of epochs (pp. 273–274). This approach ensures that the model is trained long enough to capture relevant patterns in the data while preventing excessive adaptation to noise.

Learning Rate

As explained by Goodfellow et al. (2016), the learning rate η determines the baseline magnitude of updates during optimization, which is modified by the optimizer (pp. 290–292). A learning rate that is too large may cause divergence by overshooting minima, whereas one that is too small can lead to excessively slow or premature convergence. Selecting an appropriate learning rate is, therefore, essential for stable and efficient training.

4.5 Hyperparameter Optimization

Having defined the set of potential hyperparameters, the next question that arises is how to determine the optimal topology. In most research addressing the approximation of stochastic volatility models with neural networks, the topology is chosen arbitrarily. Sala and Pravosud (2022) were the first to systematically address this problem (pp. 11–13). They implemented a Bayesian optimization algorithm, specifically the Tree-Structured Parzen Estimator (TPE) introduced by Bergstra et al. (2011).

The TPE treats hyperparameter optimization as a probabilistic modeling problem. Instead of testing the entire hyperparameter space, it models the loss function over the space and strategically samples configurations that are more likely to yield improvements.

Let the neural network $F(x \mid \omega)$, which estimates $\widetilde{IV}^{\text{NN}}$, depend on M hyperparameters, where the domain of the i -th hyperparameter is $\mathcal{H}_i$, and the joint hyperparameter space is $\mathcal{H} = \mathcal{H}_1 \times \cdots \times \mathcal{H}_M$. A specific configuration is denoted by $\mathcal{A} \in \mathcal{H}$, with $F_{\mathcal{A}}(x \mid \omega)$ representing the network using hyperparameters $\mathcal{A}$.

The loss function $l(\mathcal{A})$ is defined as the mean squared error computed on a validation subset of the training data. The model weights $\hat{\omega}$ are assumed to be perfectly calibrated for each hyperparameter choice, so the optimization reduces to comparing network topologies within $\mathcal{H}$:

$$\hat{\mathcal{A}} = \arg \min_{\mathcal{A} \in \mathcal{H}} \frac{1}{N} \sum_{i=1}^N \left(\widetilde{IV}^{\text{NN}}(\theta_i, \gamma_i; \hat{\omega}, \mathcal{A}) - \widetilde{IV}^{\text{MC}}(\theta_i, \gamma_i) \right)^2. \quad (4.5)$$

Sala and Pravosud (2022) provide a detailed description of the TPE algorithm (pp. 11–13). At each iteration, the optimizer selects a new hyperparameter configuration to maximize the Expected Improvement (EI), defined as the expected amount by which the loss function $l(\mathcal{A})$ will (negatively) exceed some threshold y^* :

$$\text{EI}_{y^*}(\mathcal{A}) = \int_{-\infty}^{\infty} \max(y^* - y, 0) p_F(y \mid \mathcal{A}) dy, \quad (4.6)$$

where $p_F(y \mid \mathcal{A})$ denotes the conditional density of the loss given $\mathcal{A}$. The threshold y^* is typically set to a quantile γ (in this case, 20%) of the observed losses y , so that $p_F(y < y^*) = \gamma$. Since $p_F(y \mid \mathcal{A})$ is difficult to estimate directly, the TPE algorithm applies Bayes' rule and instead models it as $p_F(y) p_F(\mathcal{A} \mid y)$. $p_F(\mathcal{A} \mid y)$ is defined with two densities:

$$p_F(\mathcal{A} \mid y) = \begin{cases} l(\mathcal{A}), & \text{if } y < y^*, \\ g(\mathcal{A}), & \text{if } y \geq y^*. \end{cases} \quad (4.7)$$

The expected improvement can be expressed as

$$\int_{-\infty}^{\infty} \max(y^* - y, 0) \frac{p_F(y) p_F(\mathcal{A} \mid y)}{p_F(\mathcal{A})} dy. \quad (4.8)$$

Combining these definitions with

$$p_F(\mathcal{A}) = \int_{\mathbb{R}} p_F(y) p_F(\mathcal{A} \mid y) dy = \gamma l(\mathcal{A}) + (1 - \gamma) g(\mathcal{A}), \quad p_F(y < y^*) = \gamma \quad (4.9)$$

then

$$\int_{-\infty}^{\infty} \max(y^* - y, 0) p_F(y) p_F(\mathcal{A} \mid y) dy = l(\mathcal{A}) \int_{-\infty}^{\infty} \max(y^* - y, 0) p_F(y) dy, \quad (4.10)$$

$$= \gamma y^* l(\mathcal{A}) - l(\mathcal{A}) \int_{-\infty}^{\infty} p(y) dy. \quad (4.11)$$

Finally, the order of improvement is proportional to:

$$\text{EI}_{y^*}(\mathcal{A}) = \frac{\gamma y^* l(\mathcal{A}) - l(\mathcal{A}) \int_{-\infty}^{\infty} p(y) dy}{\gamma l(\mathcal{A}) + (1 - \gamma) g(\mathcal{A})} \propto \left[\gamma + \frac{g(\mathcal{A})}{l(\mathcal{A})} (1 - \gamma) \right]^{-1} \quad (4.12)$$

The TPE maximizes EI by selecting architectures with the highest ratio $l(\mathcal{A})/g(\mathcal{A})$ at each iteration, where $l(\mathcal{A})$ and $g(\mathcal{A})$ are modeled using Parzen window estimators.

5 Methodology

5.1 Neural Network Setup

The neural network setup in this thesis follows the two-step pointwise approach. The hyperparameter search ranges are largely inspired by prior research on neural network approximations of stochastic volatility models (see Tables A.1 and A.2):

- Hidden layers: 1-5
- Nodes per layer: 10-500 (step size: 10)
- Optimizers: Adam, RMSProp
- Activation function: Softplus
- Batch sizes: 32, 64, 128, 256, 512, 1024
- Epochs: 200

Both the inputs and outputs of the neural network are scaled, as inspired by Sala and Pravosud (2022, p. 27). For the outputs, which can theoretically take values in $(0, \infty)$, standardization is applied by subtracting the mean and dividing by the standard deviation. For the inputs, which have natural bounds, MinMax scaling is used to rescale values to a consistent range of $(0, 1)$.

A learning rate scheduler is implemented to dynamically adjust the base learning rate during training (PyTorch 2.9 Documentation, n.d.-a). If the validation loss fails to improve for more than 15 epochs, the learning rate is reduced by 10%, allowing the optimizer to fine-tune updates and improve convergence.

Additionally, Batch Normalization is applied to normalize each layer’s inputs to a zero mean and unit variance. This technique enables the network to learn with higher learning rates and helps prevent issues such as exploding or vanishing gradients (Ioffe & Szegedy, 2015, pp. 1–6).

Model training is conducted in PyTorch (Paszke et al., 2019), with TPE hyperparameter optimization being performed using Optuna (Akiba et al., 2019). A total of 100 optimization trials were executed. Despite the use of an NVIDIA A5000 Pro GPU in combination with 32 CPU cores, the large training dataset resulted in a total hyperparameter optimization time of several days.

5.2 Train Data

The basis for neural network training is a Monte Carlo-generated training dataset of European payer swaptions. This dataset encompasses a large amount of data connecting a vector of SABR parameters $\theta = (\alpha, \beta, \rho, \nu)$ and market inputs $\gamma = (\tau, \frac{K}{f})$ to the implied volatility $\widetilde{IV}^{\text{MC}}$.

Counterintuitively, the forward rate f is not included as a direct input to the neural network. At first glance, this restriction might appear problematic, since for any $\beta \in (0, 1]$ the level of f affects the dynamics of the SABR process and, consequently, the implied volatility curve. However, as described by Stuijt (2021), this limitation can be resolved through a transformation that adjusts the model outputs to any forward rate (p. 37). Specifically, for a given set of parameters $\theta = (\alpha, \nu, \rho)$ and market inputs $\gamma = (K, \tau, f + \delta)$, a shift in the forward rate by δ produces the same implied volatilities as the transformed set:

$$\theta' = \left(\alpha \left(\frac{f}{f + \delta} \right)^\beta, \nu, \rho \right), \gamma' = \left(\frac{fK}{f + \delta}, \tau, f \right). \quad (5.1)$$

Thus, the model is trained at a single reference forward rate and can subsequently be extended to any other forward level through this transformation.

For the dataset, maturities are sampled from the following fixed vector:

$$\tau \in [30\text{d}, 60\text{d}, 90\text{d}, 180\text{d}, 270\text{d}, 1\text{y}, 2\text{y}, 3\text{y}, 4\text{y}, 5\text{y}, 6\text{y}, 7\text{y}, 8\text{y}, 9\text{y}, 10\text{y}],$$

using a 360-day year convention.

The forward rate is sampled from $f \sim U(0.04, 0.06)$. For training, all parameters are transformed to correspond to a single reference level using (5.1). The midpoint of the sampled range, $f = 0.05$, is chosen as the reference forward rate.

Consistent with swaption market conventions, the strikes in the training dataset are expressed in basis points relative to the forward rate. Specifically, the dataset uses strike increments of 12.5 basis points within a range of ± 250 basis points around the forward rate. Additionally, in the experiments, the forward rate equals the spot rate, as the discount rate is set to zero for simplicity.

Further, β is fixed at 0.5. To ensure comprehensive coverage across parameter combinations, the remaining parameters are drawn from uniform distributions. Parameter bounds are based on the ranges observed by Rebonato et al. (2012) and significantly widened to enhance generality (pp. 40–45):

$$\rho \sim U(-1, 0.5), \quad \alpha \sim U(0.01, 0.1), \quad \nu \sim U(0, 3).$$

Overall, 60,000 distinct SABR parameter combinations are sampled, resulting in $60,000 \times 41 = 2,460,000$ total option prices. These are converted to implied volatilities under the Black model using an algorithm from Jäckel (2015). A small subset of datapoints is removed, as some Monte Carlo simulations failed to produce valid implied volatilities for all strikes, due to no path deviating enough from the forward. Additionally, for strikes in their immediate vicinity, where Monte Carlo variance is likely excessively high, implied volatilities are also removed.

To obtain the corresponding implied volatilities for each parameter set, the Euler discretization scheme is employed. In order to minimize discretization bias, the time step size is set to $\Delta t = \frac{\tau}{1,000}$, and 1,000,000 Monte Carlo simulations are generated for each parameter combination. This is chosen in accordance with Duffie and Glynn (1995), who suggests the number of time steps in an Euler scheme should be proportional to the square root of the number of simulations (pp. 899–901).

5.3 Test Data

To evaluate the performance of the different calibration methods, a test dataset is also constructed. The use of simulated data for this purpose is justified by several factors, as also mentioned by Sala and Pravosud (2022, pp. 23–24). Firstly, historical market data, particularly for swaptions, is often expensive and difficult to obtain in large quantities. Additionally, real-world market data frequently contains noise and suffers from illiquidity, especially for deep ITM and far OTM strikes. Therefore, to meaningfully assess how well different approximation methods replicate the SABR model, it is advantageous to evaluate them against synthetic data, offering a cleaner and more controlled environment for comparisons.

To generate a test dataset of European payer swaptions, input parameters are sampled from the same ranges as those used for training. However, to avoid boundary cases, the strike vector is slightly reduced to ± 200 basis points around the forward rate.

While less precise Monte Carlo simulations are sufficient for generating the training data, since the neural network can learn the underlying function despite moderate levels of noise, higher accuracy is essential for evaluation (Sala & Pravosud, 2022, p. 25). To ensure a robust comparison of the calibration methods, the test dataset is generated using Monte Carlo simulations with $\Delta t = \frac{\tau}{1,400}$ time step size and 2,000,000 simulation paths.

Overall 10,000 distinct SABR parameter combinations are simulated, resulting in $10,000 \times 33 = 330,000$ total option prices, which are converted to Black implied volatilities using the same method as in the train dataset. Implied volatilities that are 0 or cannot be computed are removed.

6 Discussion of Results

6.1 Neural Network Architecture

A total of 100 trials are conducted for hyperparameter optimization using the TPE (see Figures A.1–A.3). The neural network performance is evaluated using the mean squared error as the loss function, with 70% of the training dataset used for training and 30% for validation.

The best performing neural network architecture identified through this process consists of five hidden layers with widths of 340, 260, 440, 380, and 190 neurons, respectively. The model uses the Softplus activation function and was trained using the RMSprop optimizer with a batch size of 512.

Based on the optimization results, several trends can be observed. Most of the sampled architectures center around a network depth of five layers, which consistently achieves the lowest validation losses. The TPE quickly converges toward these architectures, suggesting that deeper networks generally perform better than shallower ones. No clear trend is observed regarding layer width, although the best-performing model features relatively wide hidden layers.

Another notable finding is the impact of batch size on model performance. Larger batch sizes tend to yield lower MSE values, with the TPE most frequently selecting 512. The RMSprop optimizer also produces lower losses on average than the Adam optimizer, indicating its effectiveness for this task.

6.2 Monte Carlo Accuracy

To test the general accuracy of the Monte Carlo method used for the test dataset, implied volatilities generated using the same method are compared with those obtained using the Finite Difference Method (FDM), which provides highly accurate option prices. The parameter combinations, as well as the resulting implied volatilities, are taken from Choi et al. (2024, pp. 19–21), who obtained them from experiments by Antonov and Spector (2012) and Cai et al. (2017). When running the Monte Carlo simulation 50 times for each parameter combination, a very low bias and variance are observed across all scenarios (see Table B.1). This holds true even for Case III, despite the very high value of α , compared to the size of the forward f , indicating high accuracy of the Monte Carlo simulation method.

Additionally, the first 3,000 parameter combinations from the test set are re-simulated, and the corresponding implied volatilities recomputed to identify the regions of maturity and moneyness where the Monte Carlo benchmark is least accurate. Due to computational constraints, the number of re-simulations is kept relatively small. Because non-zero implied volatilities are very rare at the boundaries of the moneyness range, the median errors in those grid cells are often based on only a few observations. Nevertheless, the re-simulation results provide valuable insights into the relative accuracy of the Monte Carlo benchmark across the grid.

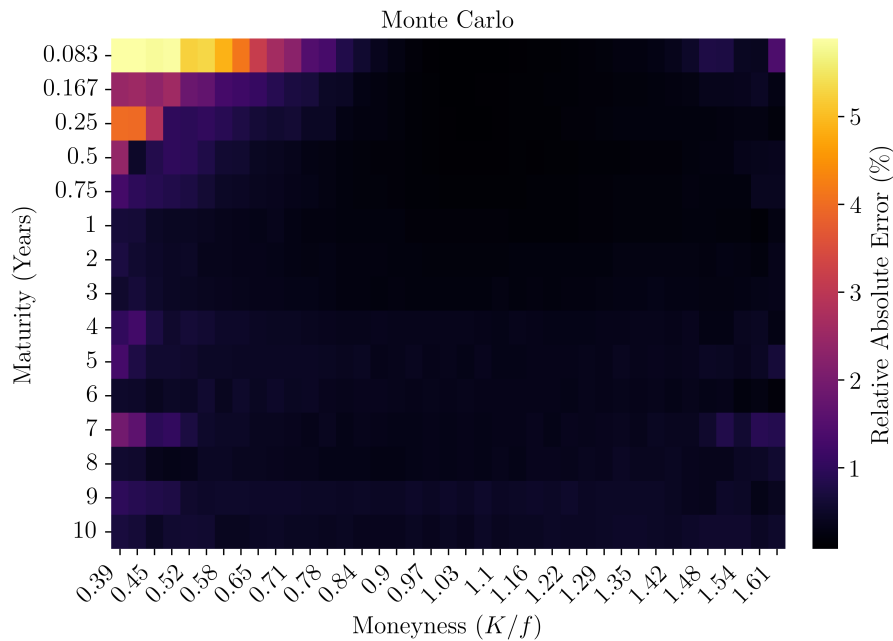


Figure 6.1: Median Monte Carlo Error

The performance of the Monte Carlo method is evaluated using the Monte Carlo error, defined as

$$\frac{|IV_1 - IV_2|}{IV_1},$$

where IV_1 is the implied volatility obtained from the first MC simulation and IV_2 denotes the implied volatility retrieved from the second MC simulation using the same parameters.

The median Monte Carlo error across all implied volatilities is 0.3%. While the median Monte Carlo error across the grid is relatively consistent, there are isolated regions where the error is comparatively high. This is mostly the case in the upper-left region of the grid, corresponding to short maturities and low strikes.

6.3 Mapping Error

In the first stage of the analysis, the goal is to measure how accurately each approximation method reproduces the implied volatility curves from the test dataset, assuming full knowledge of the real SABR parameters. The accuracy is measured using the mapping error, defined as

$$\frac{|IV_{\text{test}} - IV_{\text{map}}|}{IV_{\text{test}}},$$

where IV_{test} is the implied volatility obtained from the Monte Carlo simulations, and IV_{map} denotes the implied volatility produced by each approximation method, with knowledge of the true underlying SABR parameters.

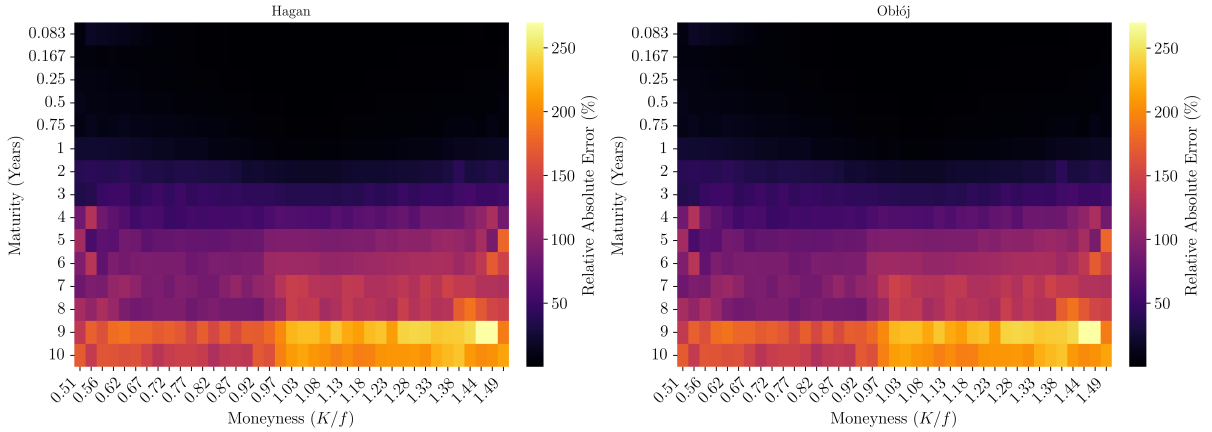


Figure 6.2: Median Mapping Error: Hagan and Obłój

Starting with the Hagan and Obłój approximations, the median mapping error for both formulas is 16.1%, indicating a substantial deviation from the Monte Carlo benchmark. Their error patterns across the grid are also virtually indistinguishable, indicating that the

refinements proposed by Obłój (2008) do not materially enhance accuracy under the tested conditions. Therefore, subsequent comparisons focus solely on the Hagan approximation.

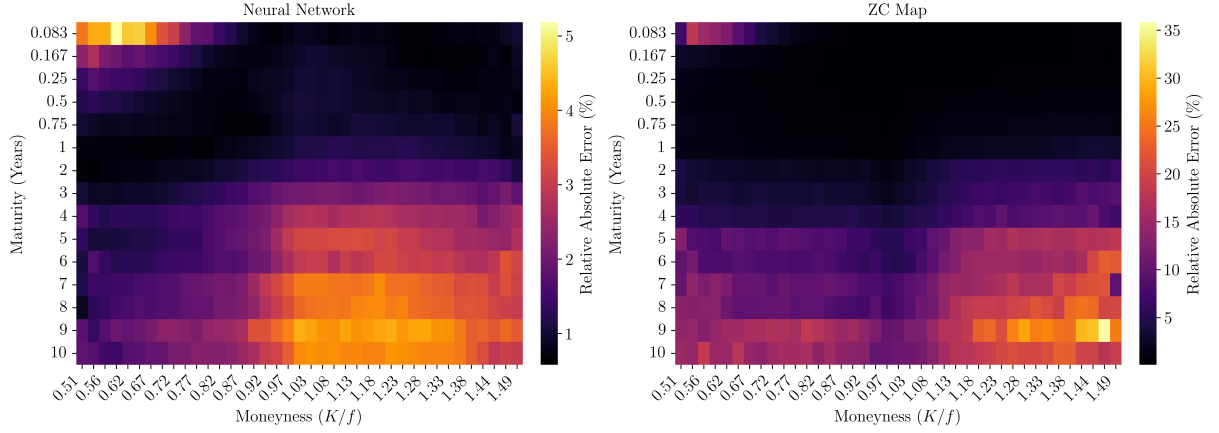


Figure 6.3: Median Mapping Error: Neural Network and ZC Map

In contrast, the neural network and ZC Map approximators demonstrate superior performance, with median mapping errors of 1.5% and 2.2%, respectively. This represents a significant improvement over the analytical formulas.

Across all approximators, a consistent pattern can be observed in how their mapping errors are distributed across the moneyness–maturity grid: mapping errors tend to increase with maturity. One possible explanation is that, for a given level of ν , a larger maturity τ amplifies the impact of the volatility of volatility ν on the implied volatility curve. Consequently, increases in both τ and ν may produce excessively steep volatility curves, which are more difficult to approximate. This hypothesis is supported by the median mapping error plotted over a grid of τ and ν , where higher values of both parameters coincide with a rise in mapping error.

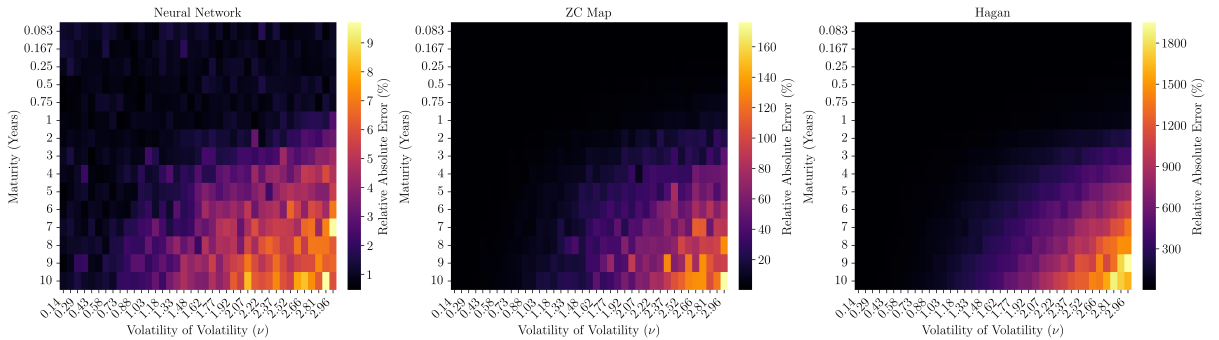


Figure 6.4: Effect of τ and ν on Median Mapping Error: Neural Network, ZC Map, and Hagan

While all methods exhibit this trend, the effect is particularly pronounced for the Hagan and ZC Map approximations. This can likely be attributed to the neural network being explicitly trained under such extreme conditions, thereby enhancing its performance in parameter regimes where the accuracy of the other approximations begins to deteriorate.

The dependence of mapping errors on moneyness, however, differs by approach. For the Hagan and Obłój formulas, the median mapping error depends primarily on maturity, showing minimal strike sensitivity. By contrast, the neural network and ZC Map approximations exhibit more pronounced error variation as a function of moneyness.

Interestingly, the ZC Map performs slightly better near the at the money region, where the median mapping error remains small regardless of maturity. This is particularly noteworthy since the ZC Map is not defined at the money. Its value is approximated by interpolating between two symmetrically chosen points, slightly above and below the at the money point.

However, the ZC Map occasionally fails to produce valid results when the term $\tilde{\nu}^2$ turns negative (see Equation D.1). These breakdowns highlight a structural limitation of the approximation and its sensitivity to extreme parameters. Points where the ZC Map failed are excluded from this chapter’s analyses.

Slightly elevated errors appear for very short maturities and deep ITM strikes across all methods, though not as visible for the Hagan and Obłój approximation due to overall elevated median errors. A comparison with the median Monte Carlo errors shows that implied volatilities in this region are highly inaccurate, suggesting that the increased mapping errors are likely driven by imperfections in the Monte Carlo benchmark rather than by deficiencies of the approximators.

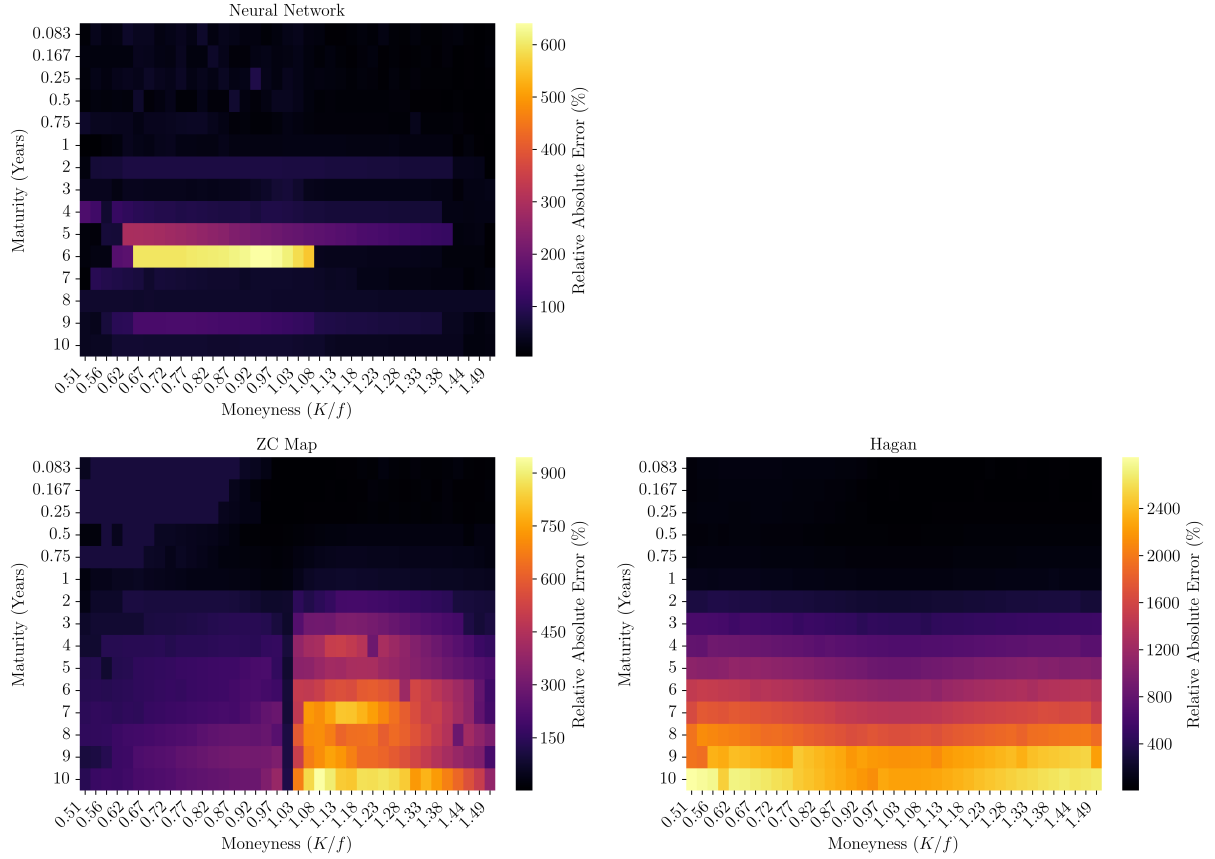


Figure 6.5: Maximum Mapping Error: Neural Network, ZC Map, and Hagan

To assess the upper bound of mapping errors, grids of the maximum mapping errors are examined. While the neural network rarely exceeds a maximum mapping error of 300%, the ZC Map can reach errors slightly above 900%, and the Hagan approximation up to approximately 2800%. Looking further into the data, the errors above 300% for the neural network are exclusively driven by a single implied volatility curve. This could raise concerns that the neural network approximation may fail due to unforeseen reasons. However, a closer inspection of the specific implied volatility curve shows that the Hagan approximation produces a comparable error, indicating that the issue is not unique to the neural network. Despite these differences in magnitude, the spatial distribution of the maximum mapping errors across the grid somewhat mirrors that of the median mapping errors for both the Hagan and ZC Map approximations.

Following the approach of Sala and Pravosud (2022), a statistical evaluation of the consistency of the three approximation methods is conducted (pp. 31–33). For this purpose, a linear regression model is estimated:

$$IV_{\text{test}} = \alpha + \beta \widetilde{IV} + \varepsilon, \quad (6.1)$$

where IV_{test} represents the Monte Carlo-generated implied volatility, and $\widetilde{IV}$ denotes the implied volatility estimated from the corresponding approximation. Standard errors are generated using the heteroscedasticity-consistent covariance matrix estimator by MacKinnon and White (1985, pp. 306–309).

As noted by Sala and Pravosud (2022), due to the very large size of the dataset, any statistical test for $\alpha = 0$ and $\beta = 1$ will likely be rejected, even when the approximations are qualitatively very close to the Monte Carlo baseline (p. 32). This is a consequence of the extremely large sample size and underlying small inconsistencies in IV_{test} caused by the Monte Carlo error. Nonetheless, valuable insights can be gained by examining the estimated regression coefficients across methods.

Model	α	β	Std. Error	R^2	p-value	N
Neural Network	-0.0049	1.0122	0.0009	0.9767	0.0000	278287
Hagan	0.2428	0.0270	0.0005	0.0090	0.0000	278287
ZC Map	0.0071	0.9239	0.0010	0.8021	0.0000	278287

Table 6.1: Regression Results Comparing Neural Network, ZC Map, and Hagan Approximations

Starting with the Hagan and ZC Map approximations, the ZC Map shows a β value of 0.924 and a negligible intercept, suggesting a slight bias toward underprediction. The regression for the Hagan approximation, on the other hand, resulted in a β value of 0.027 and an intercept value of 0.243. This highlights the limited ability of the Hagan approximation to accurately reproduce the implied volatility curves, suggesting a systematic tendency to generate overly steep curves and, consequently, inflated implied volatilities.

The neural network approximation, in contrast, performs significantly better. It achieves a β value of 1.012 and also a negligible intercept, indicating that it is nearly unbiased in reproducing the implied volatilities. Furthermore, the R^2 value for the neural network regression is the highest among all three methods. These results further validate the neural network as the most robust and reliable approximation method.

6.4 Calibration Error

In the next step, the calibration error is measured, which is defined as

$$\frac{|IV_{\text{test}} - IV_{\text{cal}}|}{IV_{\text{test}}},$$

where IV_{test} is the implied volatility obtained from Monte Carlo simulation, and IV_{cal} the error obtained when calibrating the different approximators to the implied volatility curves in the test dataset, as outlined in Section 3.4. There are a multitude of optimization algorithms that could be used for the calibration process, but based on the insights of Sala and Pravosud (2022), the TPE algorithm outperforms gradient-based algorithms such as the Trust Region method, motivating the use of the TPE in this thesis (p. 34).

The calibration is conducted as follows: first, an initial guess is provided using the formula of Floc’h and Kennedy (2014, pp. 6–7), based on the implementation of Stuijt (2021), which offers a reliable starting point for the optimization procedure. Then, the TPE optimizes the parameters to find those under which the respective approximator creates an implied volatility curve minimizing the MSE relative to the real curve. For each curve this process is repeated for 300 trials.

Method	Hagan	ZC Map	Neural Network
Average calibration time per curve (seconds)	~1.1	~42.8	~2.2

Table 6.2: Average Calibration Time per Implied Volatility Curve

All calibration experiments were conducted on a MacBook Air (Apple M3 chip and 16 GB of unified memory). As expected, there are significant differences in computational time between the various SABR approximations, with the ZC Map taking almost forty times as long as the Hagan approximation. The neural network, by contrast, requires only about twice the time of the Hagan approximation.

Examining the median calibration error across the three approximation methods, neural network, ZC Map, and Hagan, reveals that all perform comparably well. The neural network achieves a median calibration error of 1.1%, compared to 1.6% for the ZC Map and 1.8% for the Hagan approximation. This suggests that all three methods are capable of accurately replicating the implied volatility curves from the test dataset.

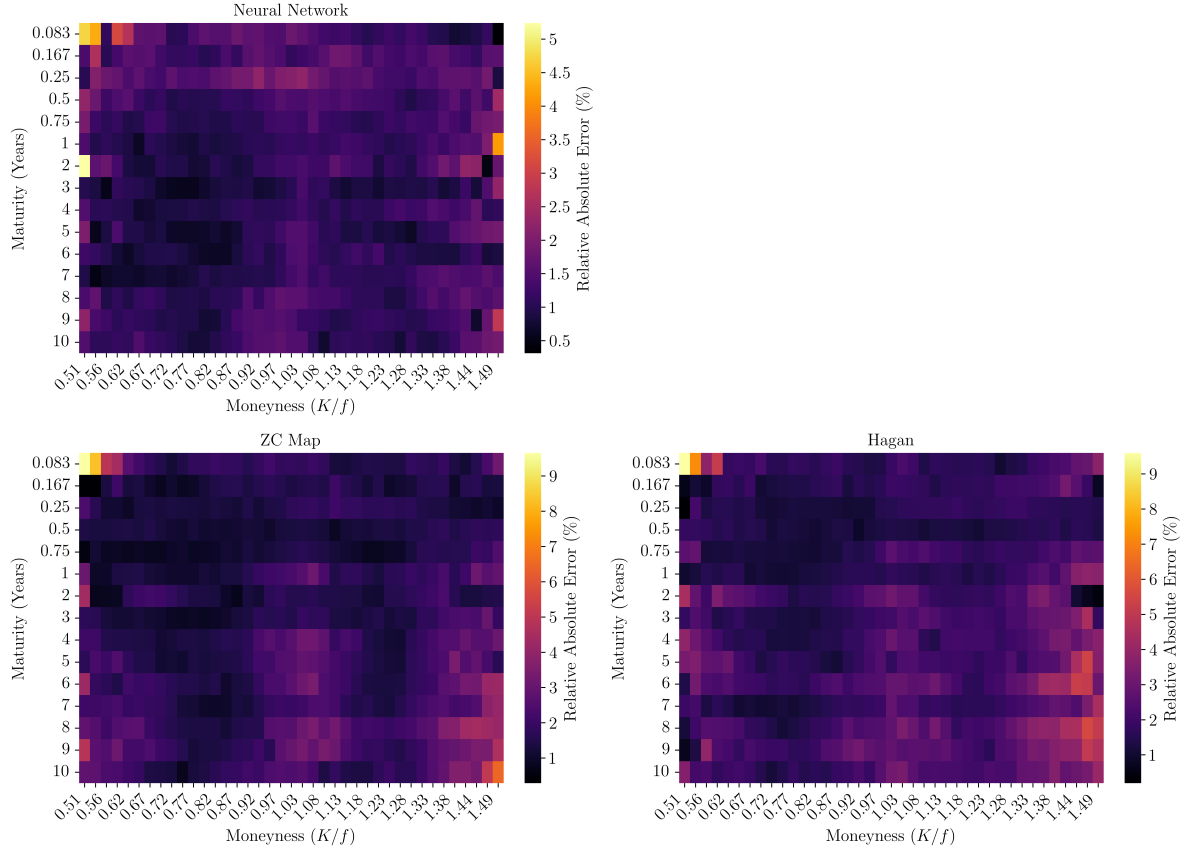


Figure 6.6: Median Calibration Error: Neural Network, ZC Map, and Hagan

However, when analyzing the median calibration error distribution across moneyness and maturity, the maximum median errors for both the Hagan and ZC Map approximations are approximately twice as large as those of the neural network. Despite this, the overall distribution of median errors across the grid remains similar. Additionally, in the region of low maturities and low moneyness, the calibration errors are noticeably higher. This is most likely again attributable to inaccuracies in the Monte Carlo benchmark implied volatilities.

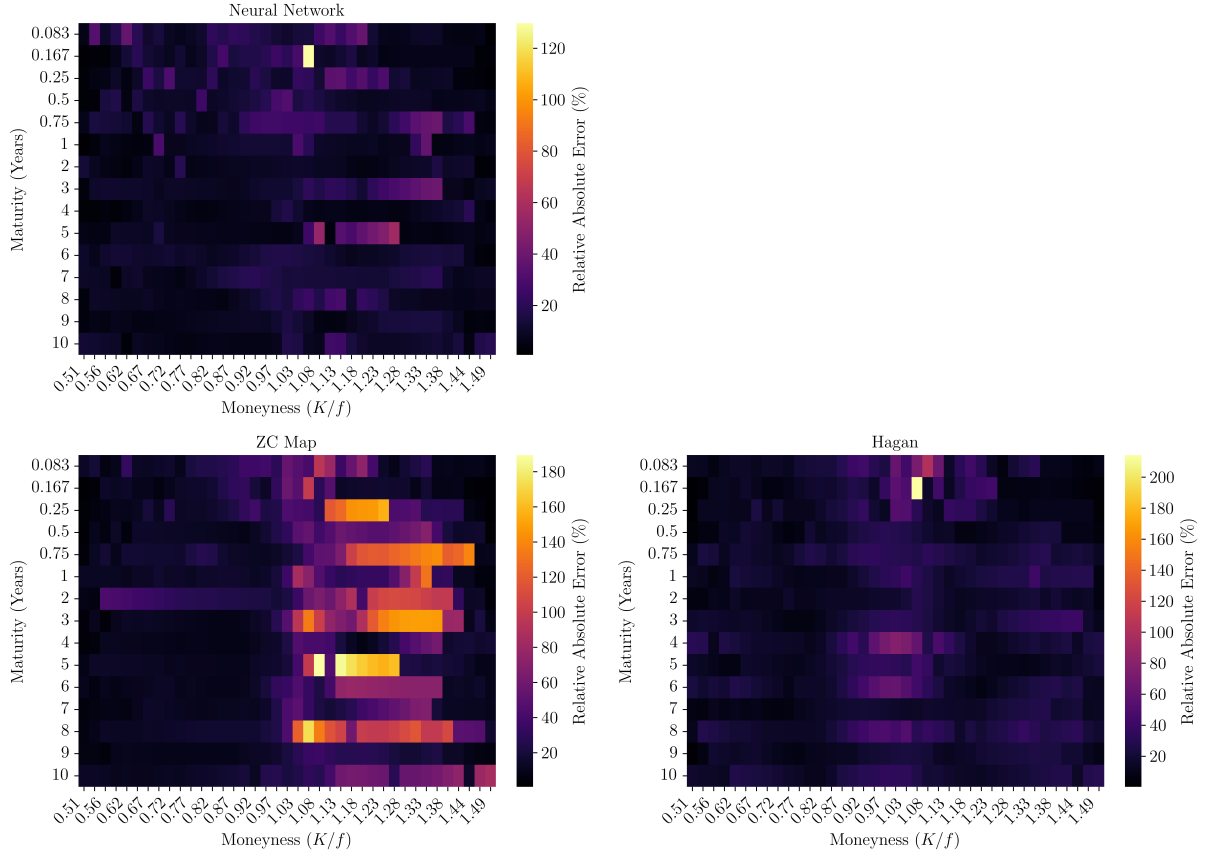


Figure 6.7: Maximum Calibration Error: Neural Network, ZC Map, and Hagan

Considering the maximum calibration errors, the neural network again performs best, with a maximum calibration error of 129.8%, compared to 189.5% for the ZC Map and 214.3% for the Hagan approximation. An interesting pattern also emerges: maximum errors are generally higher for OTM options than for ITM options, especially in the ZC Map. This could potentially stem not from the approximation techniques themselves but from the sampling characteristics of the test dataset. Since ρ is drawn from a uniform distribution $\rho \sim U(-1, 0.5)$, this could contribute to the uneven distribution of maximum calibration errors. The outlier observed in both the Hagan and neural network approximations stems from a single, unusually low implied volatility in the Monte Carlo benchmark, whose effect is amplified by the use of relative errors.

Overall, the neural network outperforms the ZC Map and Hagan approximations, demonstrating a superior ability to replicate the implied volatility curves. However, the other approximations also perform reasonably well, with both the Hagan and ZC Map methods achieving substantially better accuracy than when mapping from known parameters. Nevertheless, it remains uncertain whether the calibrated parameters align with the real parameters, since similar implied volatility curves can arise from different SABR parameter combinations. This distinction is crucial, as it directly affects the model's dynamics.

Parameter	Hagan	ZC Map	Neural Network
α	0.0042	0.0031	0.0022
ρ	0.0732	0.0873	0.0334
ν	0.5596	0.2812	0.1770

Table 6.3: Median Absolute Parameter Errors: Neural Network, ZC Map, Hagan

Table 6.3 reports the median absolute parameter error, where the parameter error and absolute parameter error, respectively, are defined as

$$\theta_{\text{test}} - \theta_{\text{cal}} \quad \text{and} \quad |\theta_{\text{test}} - \theta_{\text{cal}}|.$$

Here, θ_{test} denotes the true parameter values and θ_{cal} represents the parameters obtained during calibration using each respective approximation method. Because ρ can take positive and negative values and may approach zero, a relative error would distort the results.

It can be observed that the α parameter is the easiest to recover when compared to its sample range. However, its error distribution (see Figure C.1) is right-skewed for all approximators, indicating a tendency to largely underestimate α . In contrast, ρ and especially ν are more difficult to estimate accurately. Both the neural network and the ZC Map approximation appear mostly unbiased in estimating ρ , whereas the Hagan approximation shows an upward bias. A similar pattern as in α emerges for ν , where all three methods exhibit right-skewed error distributions, though the skew is most pronounced for the parameter errors from the Hagan approximation. This is in line with the earlier regression results: Hagan’s formula tends to produce steeper volatility smiles and most likely overstates the impact of ν .

Overall, the neural network shows the highest accuracy and the least bias in recovering the parameters α , ρ , and ν compared to the other approximation methods. This discrepancy explains the apparent improvement from mapping error to calibration error for the ZC Map, and particularly for the Hagan approximation, since the good curve fits likely result from incorrect parameter estimates. Consequently, calibration results can be misleading: even if an approximation appears to reproduce the implied volatility curve well, it does not necessarily imply that the underlying SABR parameters are correctly captured. This finding is consistent with established evidence that, although Hagan’s formula often provides an excellent fit to market data, it does not guarantee accurate parameter recovery (Sala & Pravosud, 2022, p. 37).

6.5 Hedging Error

Beyond accurate parameter recovery, it is crucial that an approximation accurately reproduces the SABR model's dynamics. These can be quantified through the SABR Greeks, which measure the sensitivity of an option price to changes in the underlying model parameters. If the model fails to reflect the true dynamics, the computation of SABR Greeks will be biased. Calculations of SABR Greeks are provided by Hagan et al. (2002, pp. 17–18):

$$\text{delta} = \frac{\partial V_{\text{call}}}{\partial f} = \frac{\partial B}{\partial \sigma_B} \cdot \frac{\partial \sigma_B(K, f; \alpha, \beta, \rho, \nu)}{\partial f} + \frac{\partial B}{\partial f}, \quad (6.2)$$

$$\text{vega} = \frac{\partial V_{\text{call}}}{\partial \alpha} = \frac{\partial B}{\partial \sigma_B} \cdot \frac{\partial \sigma_B(K, f; \alpha, \beta, \rho, \nu)}{\partial \alpha}, \quad (6.3)$$

$$\text{vanna} = \frac{\partial V_{\text{call}}}{\partial \rho} = \frac{\partial B}{\partial \sigma_B} \cdot \frac{\partial \sigma_B(K, f; \alpha, \beta, \rho, \nu)}{\partial \rho}, \quad (6.4)$$

$$\text{volga} = \frac{\partial V_{\text{call}}}{\partial \nu} = \frac{\partial B}{\partial \sigma_B} \cdot \frac{\partial \sigma_B(K, f; \alpha, \beta, \rho, \nu)}{\partial \nu}, \quad (6.5)$$

where B denotes the Black option price and σ_B the corresponding Black implied volatility for a given parameter set.

Benchmarks for these option Greeks are computed using the Monte Carlo simulation. The derivatives with respect to f , α , ρ , and ν are approximated via a central difference scheme with a relative step size of 0.25% in each direction. Estimating Greeks via Monte Carlo is typically problematic because the derivative is computed using three simulated price estimates. As a result, the variance in the simulations can dominate the true derivative signal. This issue is mitigated by, first, employing a large number of simulation paths, equal to those used for generating the test dataset, and, second, applying the common random numbers technique. The latter reduces Monte Carlo variance by using the same sequence of random numbers when determining the paths under shifted parameters that then later allow for the calculation of option prices used in the central difference approximation (Haugh, n.d., pp. 3–8). Due to computational constraints, this hedging analysis is conducted only for the first 1,000 implied volatility curves of the test dataset.

When evaluating the Greek performance for each of the approximators, the hedging error is defined as

$$|\text{Greek}_{\text{test}} - \text{Greek}_{\text{cal}}|.$$

Here, $\text{Greek}_{\text{test}}$ is the value of the SABR Greek calculated using the Monte Carlo simulation, while $\text{Greek}_{\text{cal}}$ is the value of the Greek retrieved from every one of the approximators using the SABR parameters obtained from the calibration process.

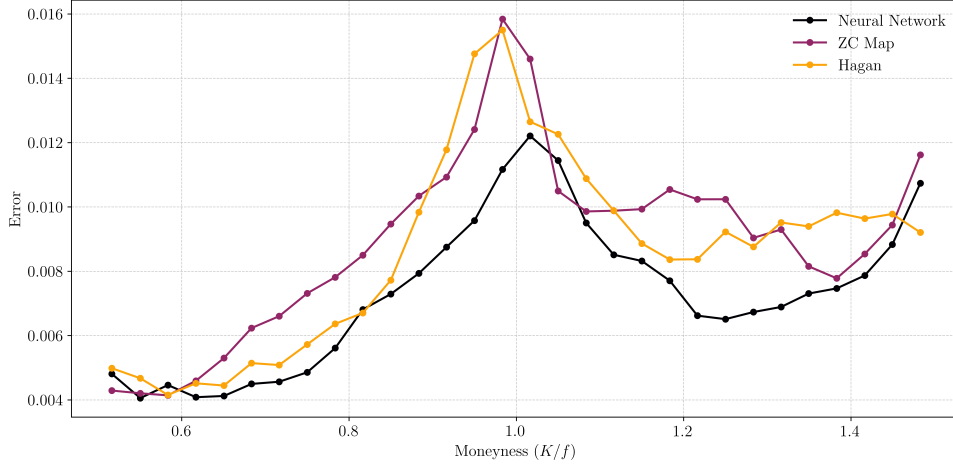


Figure 6.8: Median Delta Error: Neural Network, ZC Map, and Hagan

Examining the delta error first, the overall median hedging errors are 8.6×10^{-3} , 9.1×10^{-3} , and 7.3×10^{-3} for the Hagan, ZC Map, and neural network approximations, respectively. It can be expected that errors are low and similar across all approximators, as delta is strongly affected by the slope of the implied volatility curve. Determining the correct curve fit was the central objective of the calibration procedure. Recalling the calibration results, both Hagan and ZC Map performed comparably, whereas the neural network achieved a slightly lower median calibration error. The main delta hedging errors occur at the money and far out of the money for all three approximators.

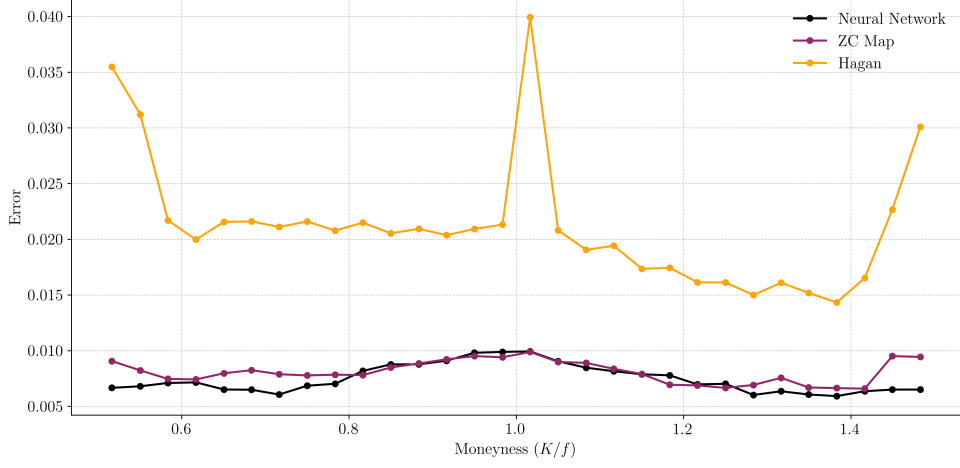


Figure 6.9: Median Vega Error: Neural Network, ZC Map, and Hagan

For vega, the median hedging errors are 2.0×10^{-2} , 8.1×10^{-3} , and 7.6×10^{-3} for the Hagan, ZC Map, and neural network approximations, respectively. The comparatively high hedging error of the Hagan approximation could possibly be attributed to inaccuracies in estimating ν during calibration. Since ν amplifies the propagation of volatility, errors in its estimation affect $\frac{\partial \sigma_B(K, f; \alpha, \beta, \rho, \nu)}{\partial \alpha}$. Vega hedging errors for the neural network and ZC Map approximations remain stable across moneyness, whereas they rise in extreme regions for the Hagan approximation, with the latter also showing a pronounced spike at the money.

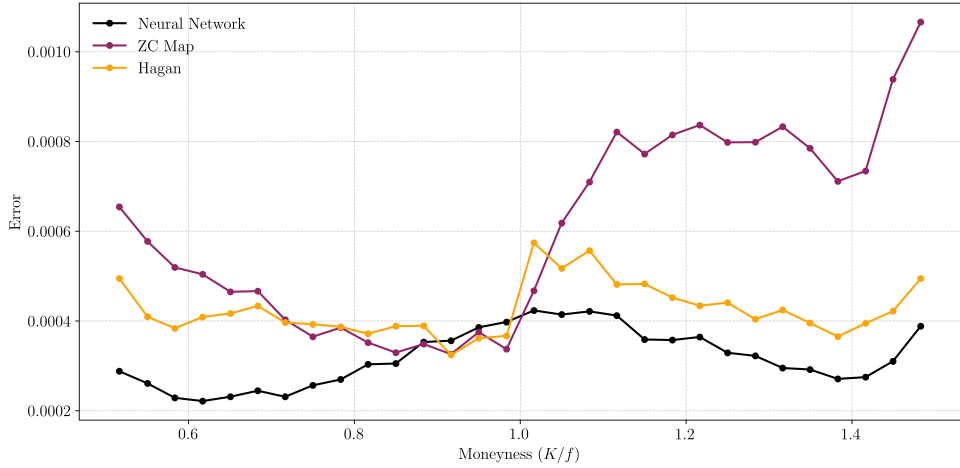


Figure 6.10: Median Vanna Error: Neural Network, ZC Map, and Hagan

For vanna, the neural network achieves a median hedging error of 3.2×10^{-4} , compared with 4.2×10^{-4} under the Hagan and 5.2×10^{-4} under the ZC Map approximation. Further analysis indicates that, in the overall median, all approximators tend to underestimate vanna. This is especially true for the Hagan approximation, which performs poorly out

of the money. Both ZC Map and neural network improve on this behavior, particularly the latter, though a slight median underestimation persists.

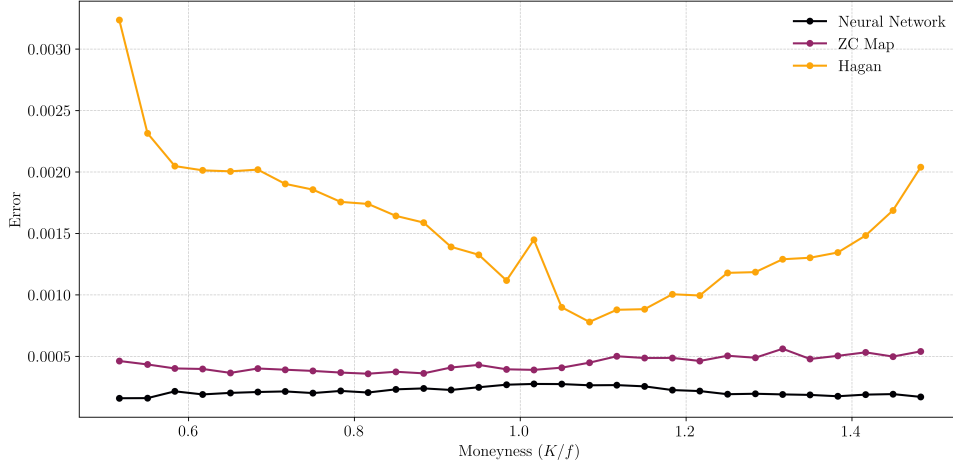


Figure 6.11: Median Volga Error: Neural Network, ZC Map, and Hagan

Lastly, considering the volga error, the median errors are 1.4×10^{-3} , 4.3×10^{-4} , and 2.2×10^{-4} for the Hagan, ZC Map, and neural network approximations, respectively. Both the ZC Map and the neural network achieve consistently low errors across the entire moneyness spectrum. The Hagan approximation, however, performs noticeably worse, particularly for deep in or out of the money options. A closer inspection of the results shows that Hagan substantially overestimates volga and thus the influence of ν , consistent with previous observations.

Overall, the neural network demonstrates the lowest and, by far, the most stable hedging errors across all four Greeks. This likely stems from its improved ability to recover accurate SABR parameters during calibration in connection with its enhanced capacity to capture the true dynamics of the SABR model.

6.6 Arbitrage Violation

The arbitrage constraints I-III (see 3.10-3.12) represent essential criteria for evaluating any SABR approximation. To assess them, the calibrated implied volatility curves for each approximation are tested, and violations are counted. For constraint I, the violating strike itself is assigned the violation. For constraint II, violations are assigned to the point with the higher strike, and for constraint III, the midpoint strike is assigned the violation. In the analysis of relative constraint III violations, the highest moneyness strike was removed, since, by the given definition, it cannot violate the butterfly arbitrage condition.

Constraint I, the positivity of option prices, is satisfied by all methods. For the neural network, this is ensured by the Softplus activation function in the output layer, which prevents negative implied volatilities. For the ZC Map, option prices are computed first and then converted into implied volatilities. In the case of a negative option price, the calibration trial would fail, since the implementation of the Black model approximation of Jäckel (2015) does not accept negative inputs.

Approximation	Constraint II		Constraint III	
	No tolerance	0.5% tolerance	No tolerance	0.5% tolerance
Hagan	0.04%	0.04%	0.70%	0.37%
ZC Map	0.08%	0.08%	0.56%	0.30%
Neural Network	0.12%	0.09%	2.81%	0.37%

Table 6.4: Arbitrage Violations Across Approximation Methods

For the vertical spread constraint, violations are rare across all three approximation methods. In contrast, violations of the butterfly spread constraint are considerably more frequent. The neural network, in particular, exhibits a comparatively high share of violations, with 2.81% of prices breaching condition III. This represents a meaningful shortcoming of the neural network approximation. Introducing a small tolerance of 0.5% to the price reduces the violation rate to 0.37%, a much stronger reduction than observed for the other approximation methods. This indicates that, although the neural network produces more violations, these tend to be less severe in magnitude than those of the other approximators.

The reason for the high arbitrage violation rate of the neural network remains unclear. One possibility is that the effect is inherited from the training data. The Monte Carlo dataset used for training is not perfectly arbitrage-free due to discretization, and the

neural network may have learned these inconsistencies. Therefore, a repeated training run was conducted, where all arbitrage-violating strikes were removed from the training set. This, however, did not materially reduce the number of arbitrage violations in the calibrated curves of the neural network. A supplementary explanation lies in the pointwise training approach. Since the network is trained on individual implied volatilities rather than full curves, it never explicitly learns the relationship between neighboring strikes.

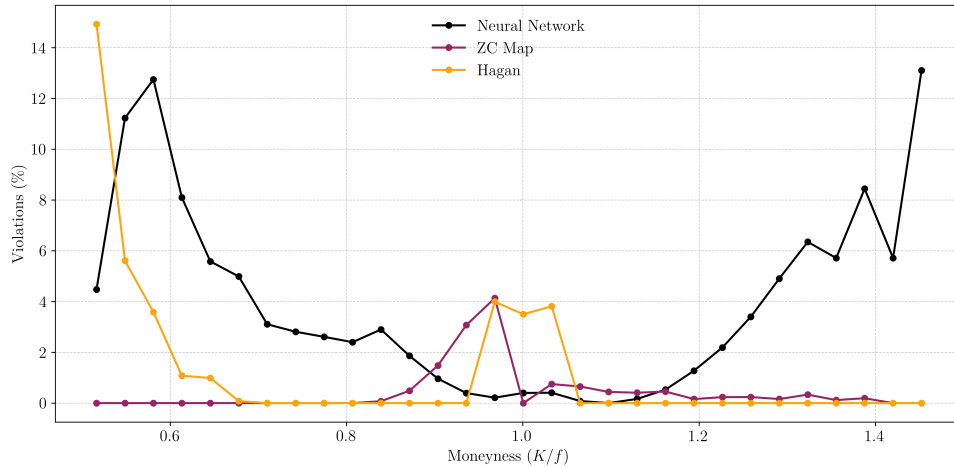


Figure 6.12: Butterfly Arbitrage Constraint Violations vs. Moneyness: Neural Network, ZC Map, and Hagan

When examining butterfly arbitrage violations as a function of maturity, the neural network exhibits violations predominantly in deep in or out of the money regions. For the Hagan and ZC Map approximations, the pattern differs. A lot of arbitrage violations occur at or around the money. Beyond this region, the ZC Map approximation's share of arbitrage violations vanishes almost completely, whereas the share of Hagan approximation violations gets very large deep in the money. Additionally, almost all butterfly arbitrage violations across methods happen for short maturities, $\tau < 2$.

Since the butterfly arbitrage violations pose a concern for the practical applicability of the neural network approximation, potential mitigation strategies should be considered. One possibility is to incorporate an explicit penalty for arbitrage violations into the neural network's loss function, as proposed by Itkin (2019, pp. 7–9). Although this method significantly reduces out-of-sample arbitrage violations, Itkin (2019) notes that it does not eliminate them entirely and may slightly degrade overall performance. Additionally, the approach may be more challenging to apply in the context of this paper, as the model is trained to output single implied volatilities rather than a grid of option prices, where such a penalty term would be much simpler to implement.

7 Conclusion

This thesis examined the use of neural networks as an alternative approximation method for the SABR model, with a particular focus on use in swaption markets. Implementing the two-step pointwise approach, the neural network demonstrated a strong performance. Across nearly all evaluation metrics, the neural network approximation outperformed both the Hagan and Zero Correlation Map approximations. It produced more accurate implied volatilities for given SABR parameters, achieved a closer fit to observed volatility curves during calibration, and delivered more precise estimates of the underlying SABR parameters. Moreover, the network captured the dynamics of the SABR model remarkably well, providing consistent and accurate SABR Greeks. All this while achieving computational efficiency close to Hagan’s formula.

Unfortunately, the neural network approximation also exhibits several limitations that must be acknowledged. The most significant concern is its tendency to violate arbitrage constraints, which poses a serious obstacle to practical adoption. One possible reason is the naive Euler discretization Monte Carlo simulation for generating the training data, which may introduce arbitrage violations that the network subsequently learns. Future research should therefore investigate arbitrage behavior under different schemes. Moreover, violations may be further reduced by the technique discussed in Section 6.6. Incorporating arbitrage constraints directly into the loss function during neural network training represents a promising direction for reducing out-of-sample violations.

Additionally, while differences in performance across the three approximators were substantial under the parameter ranges used in this study, these differences are expected to be less pronounced under real market conditions. As shown in Figure 6.4, in regimes of high ν and high τ , both the Hagan and ZC Map approximations lose accuracy, whereas the neural network, trained explicitly on such combinations, maintains a comparatively better performance. Empirical findings by Rebonato et al. (2012, pp. 41–43) and intuition from McGhee (2018, p. 11), however, indicate that ν tends to decline with maturity. Consequently, in practice, the difference in errors of the neural network and other approx-

imation methods is likely smaller than under the sample ranges in the test dataset. This would also warrant further research on comparing the neural network SABR approximation with other approximation methods based on real swaption market data, providing a clearer picture of whether the advantages observed in the synthetic test environment persist in a real-world setting.

A further concern about the neural network approximation is interpretability. The neural network's internal decision-making process is a black box that cannot be easily analyzed or validated. This opacity represents a material constraint for adoption in real markets, where model transparency can be important. The isolated, but relatively severe, miscalculation observed for an individual implied volatility curves (see Figure 6.5) underscores the relevance of this concern.

Despite these limitations, the results of this thesis show that neural networks constitute a highly accurate and computationally efficient alternative to existing SABR approximation methods. When trained on a precise and extensive dataset, neural networks can reproduce the SABR mapping with great accuracy while also offering computational efficiency, making them a promising direction for swaption volatility modeling.

Bibliography

- Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2623–2631. <https://doi.org/10.1145/3292500.3330701>
- Antonov, A., Konikov, M., & Spector, M. (2013). Sabr spreads its wings. *Risk Magazine*, 58–63. <https://www.numerix.com/resources/quantitative-research/sabr-spreads-its-wings>
- Antonov, A., Konikov, M., & Spector, M. (2015). The free boundary sabr: Natural extension to negative rates. *SSRN Electronic Journal*. <https://doi.org/10.2139/SSRN.2557046>
- Antonov, A., Konikov, M., & Spector, M. (2019). *Modern sabr analytics: Formulas and insights for quants, former physicists and mathematicians*. Springer. <https://doi.org/10.1007/978-3-030-10656-0>
- Antonov, A., & Spector, M. (2012). Advanced analytics for the sabr model. *SSRN Electronic Journal*. <https://doi.org/10.2139/SSRN.2026350>
- Bayer, C., Horvath, B., Muguruza, A., Stemper, B., & Tomas, M. (2019). On deep calibration of (rough) stochastic volatility models. *arXiv*. <https://arxiv.org/abs/1908.08806>
- Berestycki, H., Busca, J., & Florent, I. (2004). Computing the implied volatility in stochastic volatility models. *Communications on Pure and Applied Mathematics*, 57(10), 1352–1373. <https://doi.org/10.1002/cpa.20039>
- Bergstra, J., Bardenet, R., Bengio, Y., & Kégl, B. (2011). Algorithms for hyper-parameter optimization. *Advances in Neural Information Processing Systems 24: Proceedings of the 25th International Conference on Neural Information Processing Systems*, 2546–2554. https://proceedings.neurips.cc/paper_files/paper/2011/file/86e8f7ab32cfd12577bc2619bc635690-Paper.pdf
- Cai, N., Song, Y., & Chen, N. (2017). Exact simulation of the sabr model. *Operations Research*, 65(4). <https://doi.org/10.1287/opre.2017.1617>

- Chen, B., Oosterlee, C. W., & Weide, H. V. D. (2012). A low-bias simulation scheme for the sabr stochastic volatility model. *International Journal of Theoretical and Applied Finance*, 15(2). <https://doi.org/10.1142/S0219024912500161>
- Choi, J., Hu, L., & Kwok, Y. K. (2024). Efficient simulation of the sabr model. *arXiv*. <https://arxiv.org/pdf/2408.01898>
- Choi, J., Hu, L., & Kwok, Y. K. (2025, March 19). Efficient simulation of the sabr model [Workshop presentation]. https://www.bayes.citystgeorges.ac.uk/___data/assets/pdf_file/0003/866334/Jaehyuk-Choi-MC-Slides-Bayes.pdf
- Choi, J., & Wu, L. (2021). The equivalent constant-elasticity-of-variance (cev) volatility of the stochastic-alpha-beta-rho (sabr) model. *Journal of Economic Dynamics and Control*, 128, 104143. <https://doi.org/10.1016/j.jedc.2021.104143>
- Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems*, 2, 303–314. <https://doi.org/10.1007/BF02551274/METRICS>
- Duffie, D., & Glynn, P. (1995). Efficient monte carlo simulation of security prices. *The Annals of Applied Probability*, 5(4), 897–905. https://www-leland.stanford.edu/~duffie/jstorlinks/d_glynn1.pdf
- Floc'h, F. L., & Kennedy, G. J. (2014). Explicit sabr calibration through simple expansions. *SSRN Electronic Journal*. <https://doi.org/10.2139/SSRN.2467231>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press. <http://www.deeplearningbook.org>
- Goosen, J. (2022). *Volatility model pricing and calibration with neural networks using bayesian optimisation* [Master's thesis, University of Cape Town]. <https://open.uct.ac.za/server/api/core/bitstreams/74882897-4cd4-45f5-aea0-16a52d54b89f/content>
- Hagan, P., Kumar, D., Lesniewski, A., & Woodward, D. (2002). Managing smile risk. *Wilmott Magazine*, 1, 84–108. https://www.researchgate.net/publication/235622441_Managing_Smile_Risk
- Haugh, M. (n.d.). IEOR E4703: Monte-carlo simulation - estimating the greeks [Lecture slides]. https://www.columbia.edu/~mh2078/MonteCarlo/MCS_Greeks_MasterSlides.pdf
- Henry-Labordère, P. (2008). *Analysis, geometry, and modeling in finance: Advanced methods in option pricing* (1st ed.). Chapman and Hall/CRC. <https://doi.org/10.1201/9781420087000>
- Hernandez, A. (2017). Model calibration with neural networks. *RiskNet*. <https://www.risk.net/media/download/952386/download>

- Horvath, B., Muguruza, A., & Tomas, M. (2021). Deep learning volatility: A deep neural network perspective on pricing and calibration in (rough) volatility models. *Quantitative Finance*, 21(1), 11–27. <https://doi.org/10.1080/14697688.2020.1817974>
- Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In F. Bach & D. Blei (Eds.), *Proceedings of the 32nd International Conference on Machine Learning* (pp. 448–456, Vol. 37). PMLR. <https://proceedings.mlr.press/v37/ioffe15.html>
- Itkin, A. (2019). Deep learning calibration of option pricing models: Some pitfalls and solutions. *arXiv*. <https://arxiv.org/abs/1906.03507>
- Jäckel, P. (2015). Let’s be rational. *Wilmott*, 2015(75), 40–53. <https://doi.org/10.1002/wilm.10395>
- Karlsmark, M. (2013, July). *Four essays in quantitative finance* [Doctoral dissertation, University of Copenhagen]. <https://noter.math.ku.dk/phd13mk.pdf>
- Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. In Y. Bengio & Y. LeCun (Eds.), *3rd International Conference on Learning Representations*. <http://arxiv.org/abs/1412.6980>
- Liu, J., Zhang, T., & Sun, S. (2024). Review of deep learning algorithms in molecular simulations and perspective applications on petroleum engineering. *Geoscience Frontiers*, 15(2), 101735. <https://doi.org/10.1016/J.GSF.2023.101735>
- Liu, S., Borovykh, A., Grzelak, L. A., & Oosterlee, C. W. (2019). A neural network-based framework for financial model calibration. *Journal of Mathematics in Industry*, 9(1), 9. <https://doi.org/10.1186/S13362-019-0066-7>
- Lu, Z., Pu, H., Wang, F., Hu, Z., & Wang, L. (2017). The expressive power of neural networks: A view from the width. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, & R. Garnett (Eds.), *Advances in Neural Information Processing Systems* (Vol. 30). Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/32cbf687880eb1674a07bf717761dd3a-Paper.pdf
- Luo, C. (2016). *On the calibration of the sabr model and its extensions* [Master’s thesis, Imperial College]. https://www.imperial.ac.uk/media/imperial-college/faculty-of-natural-sciences/departments/math-finance/Cheng_Luo-thesis.pdf
- MacKinnon, J. G., & White, H. (1985). Some heteroskedasticity-consistent covariance matrix estimators with improved finite sample properties. *Journal of Econometrics*, 29(3), 305–325. [https://doi.org/10.1016/0304-4076\(85\)90158-7](https://doi.org/10.1016/0304-4076(85)90158-7)
- Malliaris, M., & Salchenberger, L. (1993). A neural network model for estimating option prices. *Applied Intelligence*, 3(3), 193–206. <https://doi.org/10.1007/BF00871937>

- McGhee, W. A. (2018). An artificial neural network representation of the sabr stochastic volatility model. *SSRN Electronic Journal*. <https://doi.org/10.2139/SSRN.3288882>
- Obłój, J. (2008). Fine-tune your smile: Correction to hagan et al. *Wilmott Magazine*, 2008. <https://arxiv.org/abs/0708.0998>
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., ... Chintala, S. (2019). Pytorch: An imperative style, high-performance deep learning library. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, & R. Garnett (Eds.), *Advances in Neural Information Processing Systems* (Vol. 32). Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2019/file/bdbca288fee7f92f2bfa9f7012727740-Paper.pdf
- Paulot, L. (2015). Asymptotic implied volatility at the second order with application to the sabr model. In P. K. Friz, J. Gatheral, A. Gulisashvili, A. Jacquier, & J. Teichmann (Eds.), *Large Deviations and Asymptotic Methods in Finance* (pp. 37–69). Springer International Publishing. https://link.springer.com/chapter/10.1007/978-3-319-11605-1_2
- PyTorch 2.9 Documentation. (n.d.-a). *ReduceLROnPlateau*. Retrieved August 26, 2025, from https://pytorch.org/docs/stable/generated/torch.optim.lr_scheduler.ReduceLROnPlateau.html#torch.optim.lr_scheduler.ReduceLROnPlateau
- PyTorch 2.9 Documentation. (n.d.-b). *RMSprop*. Retrieved August 26, 2025, from <https://docs.pytorch.org/docs/stable/generated/torch.optim.RMSprop.html>
- PyTorch 2.9 Documentation. (n.d.-c). *Softplus*. Retrieved August 26, 2025, from <https://docs.pytorch.org/docs/stable/generated/torch.nn.Softplus.html>
- Rebonato, R., McKay, K., & White, R. (2012). The sabr model. In *The sabr/libor market model* (pp. 25–50). John Wiley & Sons, Ltd. <https://doi.org/https://doi.org/10.1002/9781119206392.ch3>
- Roppel, A., Unger, N., & Uhrig-Homburg, M. (2023). Option-implied lower bound beliefs and the impact of negative interest rates. *SSRN Electronic Journal*. <https://doi.org/10.2139/SSRN.4513913>
- Sala, C., & Pravosud, M. (2022). Smart deep learning calibration of the sabr model [Accessed: 2025-2-4. Download inactive on 2025-11-9.]. *SSRN Electronic Journal*. <https://papers.ssrn.com/abstract=4083860>
- Stuijt, H. (2021). *Deep calibration of sabr stochastic volatility* [Master's thesis, Erasmus School of Economics]. https://thesis.eur.nl/pub/59437/Hugo_Stuijt_433154_thesis_final_version.pdf

- Telgarsky, M. (2016). Benefits of depth in neural networks. In V. Feldman, A. Rakhlin, & O. Shamir (Eds.), *29th Annual Conference on Learning Theory* (pp. 1517–1539, Vol. 49). PMLR. <https://proceedings.mlr.press/v49/telgarsky16.html>
- Xiong, C. (2011). Calibration of sabr stochastic volatility model. <https://modelmania.github.io/main/>

Appendix A: Hyperparameter Optimization

Hyperparameter	Bayer et al. (2019)	McGhee (2018)	Stuijt (2021)
Hidden layers	3	1	4
Nodes	30	250, 500, 750, 1000	30
Optimizer	Adam	Adam	Adam
Minibatch size	Not specified	Not specified	2048
Activation function	ELU	Softplus	ELU
Training epochs	Not specified	Not specified	200 (early stop: 25)

Table A.1: Hyperparameter Ranges in Previous Literature (Table 1 of 2)

Hyperparameter	Sala and Pravosud (2022)	Goosen (2022)
Hidden layers	1-5	1-5
Nodes	10-510 (step size: 10)	5-512 (step size: 1)
Optimizer	Adam, RMSProp	Adam
Minibatch size	128, 256, 512, 1024	32-512 (step size: 1)
Activation function	softplus	ReLU
Training epochs	300 (early stop: 20)	250 (early stop: 40)

Table A.2: Hyperparameter Ranges in Previous Literature (Table 2 of 2)

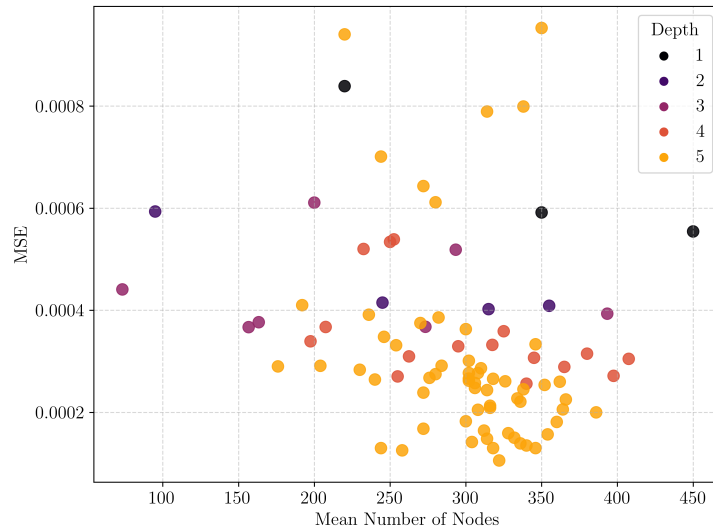


Figure A.1: Hyperparameter Optimization Results: Number of Hidden Layers & Mean Number of Nodes vs. MSE

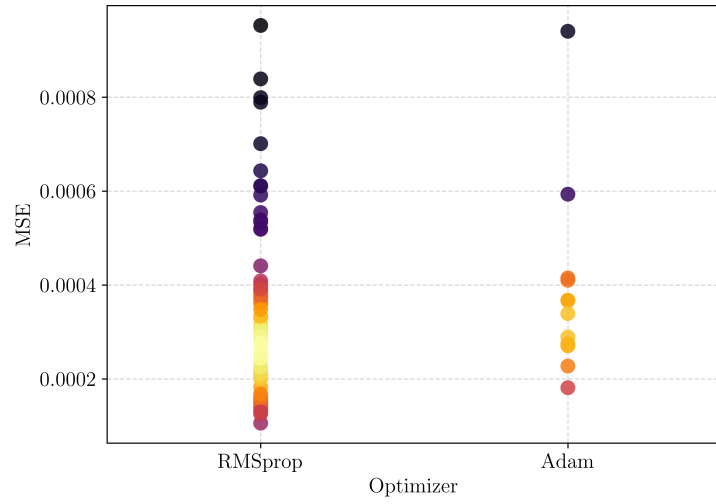


Figure A.2: Hyperparameter Optimization Results: Optimizer vs. MSE

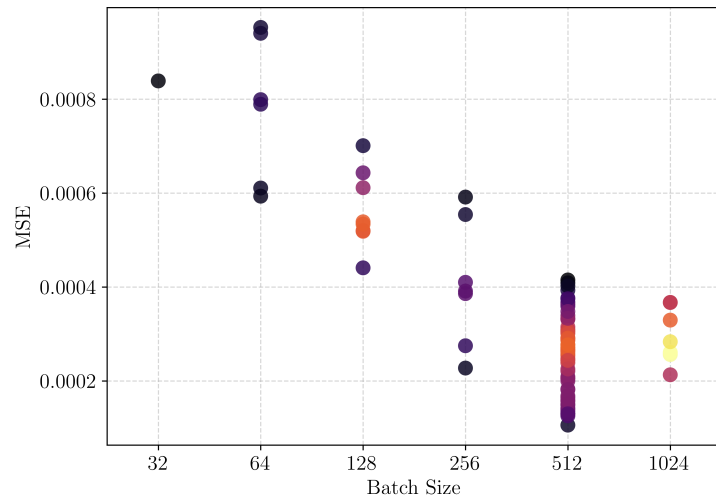


Figure A.3: Hyperparameter Optimization Results: Batch Size vs. MSE

Appendix B: Monte Carlo vs. FDM Accuracy

Case I: $f = 1, \alpha = 0.25, \nu = 0.3, \rho = -0.8, \beta = 0.3, T = 10$							
K/f	0.2	0.4	0.8	1.0	1.2	1.6	2.0
Exact implied volatility from FDM							
FDM IV	0.4828	0.3823	0.2695	0.2310	0.1994	0.1536	0.1303
Bias of MC simulation method ($\times 10^{-3}$)							
Bias	4.40	1.50	0.40	0.30	0.20	0.04	0.03
Variance of MC simulation method ($\times 10^{-6}$)							
Var	2.33	0.44	0.07	0.04	0.03	0.02	0.02

Case II: $f = 1, \alpha = 0.25, \nu = 0.3, \rho = -0.5, \beta = 0.6, T = 10$							
K/f	0.2	0.4	0.8	1.0	1.2	1.6	2.0
Exact implied volatility from FDM							
FDM IV	0.4237	0.3427	0.2607	0.2362	0.2183	0.1974	0.1892
Bias of MC simulation method ($\times 10^{-2}$)							
Bias	6.73	3.02	1.04	0.68	0.40	-0.07	-0.19
Variance of MC simulation method ($\times 10^{-6}$)							
Var	4.41	0.81	0.14	0.08	0.06	0.04	0.04

Case III: $f = 0.05, \alpha = 0.4, \nu = 0.6, \rho = 0.0, \beta = 0.3, T = 1$						
K/f	0.02	0.04	0.05	0.06	0.08	0.10
Exact implied volatility from FDM						
FDM IV	2.9223	2.6064	2.4984	2.4079	2.2638	2.1511
Bias of MC simulation method ($\times 10^{-2}$)						
Bias	14.79	6.89	5.40	4.45	3.20	2.42
Variance of MC simulation method ($\times 10^{-4}$)						
Var	8.21	1.95	1.26	0.89	0.52	0.35

Table B.1: Monte Carlo Simulation Bias and Variance Compared to Finite Difference Method (50 Trials)

Appendix C: SABR Parameter Recovery

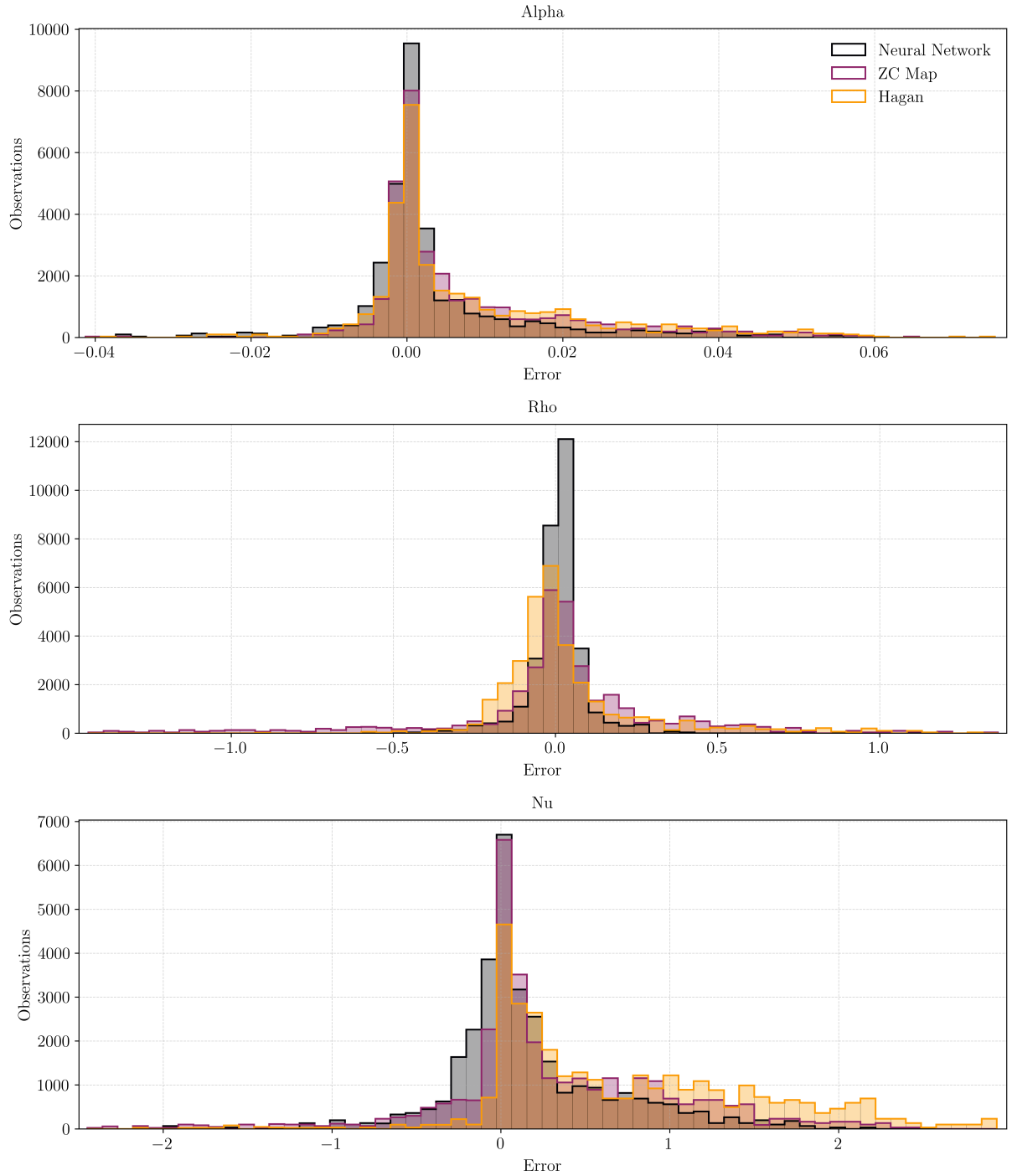


Figure C.1: Calibrated Parameter Error Distribution: Neural Network, ZC Map, and Hagan

Appendix D: Antonov et al. (2013) Approximation

The following describes the process of obtaining the option price using the ZC Map of Antonov et al. (2013, pp. 2–5). The implementation follows the procedure outlined by Stuijt (2021, pp. 55–58). For the mimicking SABR model, with 0 correlation, the parameters $\tilde{\alpha}$, $\tilde{\beta}$, and $\tilde{\nu}$ serve as the counterparts to the original SABR parameters α , β , and ν :

$$\begin{aligned}\tilde{\beta} &= \beta, \\ \tilde{\nu} &= \sqrt{\nu^2 - \frac{3}{2}\{\nu^2\rho^2 + \alpha\nu\rho(1-\beta)f^{\beta-1}\}}, \\ \tilde{\alpha} &= \tilde{\alpha}^{(0)} + \tau\tilde{\alpha}^{(1)}.\end{aligned}\tag{D.1}$$

where

$$\begin{aligned}\tilde{\alpha}^{(0)} &= \frac{2\Phi\delta\tilde{q}\tilde{\nu}}{\Phi^2 - 1}, \\ \delta\tilde{q} &= \frac{K^{1-\tilde{\beta}} - f^{1-\tilde{\beta}}}{1 - \tilde{\beta}}, \\ \Phi &= \left(\frac{\alpha_{\min} + \rho\alpha + \nu\delta q}{(1 + \rho)\alpha} \right)^{\frac{\tilde{\nu}}{\nu}}.\end{aligned}\tag{D.2}$$

with

$$\begin{aligned}\alpha_{\min} &= \sqrt{\nu^2\delta q^2 + 2\rho\nu\delta q\alpha + \alpha^2}, \\ q &= \frac{K^{1-\beta}}{1 - \beta}, \\ \delta q &= \frac{K^{1-\beta} - f^{1-\beta}}{1 - \beta}.\end{aligned}\tag{D.3}$$

The term $\tilde{\alpha}^{(1)}$ is not equal to the equation for $\tilde{\alpha}^{(1)}$ from Antonov et al. (2013). As Stuijt (2021, p. 35) points out, it is not possible to replicate the original experiments in Antonov et al. (2013) using Equation (20) in the paper. However, Equation (26) from Antonov et al. (2015) can replicate the results:

$$\tilde{\alpha}^{(1)} = \tilde{\alpha}^{(0)} \tilde{\nu}^2 \frac{\sqrt{1 + \tilde{R}^2} \frac{1}{2} \ln \left(\frac{\alpha \alpha_{\min}}{\tilde{\alpha}^{(0)} \alpha_{\min}} \right) - \mathcal{B}_{\min}}{\tilde{R} \ln(\sqrt{1 + \tilde{R}^2} + \tilde{R})}, \quad \text{for } \tilde{R} = \frac{\delta q \tilde{\nu}}{\tilde{\alpha}^{(0)}}, \quad (\text{D.4})$$

where

$$\begin{aligned} \tilde{\alpha}_{\min} &= \sqrt{\tilde{\nu}^2 \delta q^2 + (\tilde{\alpha}^{(0)})^2}, \\ \mathcal{B}_{\min} &= -\frac{1}{2} \frac{\beta}{1 - \beta} \frac{\rho}{\sqrt{1 - \rho^2}} (\pi - \varphi_0 - \arccos \rho - I), \\ \varphi_0 &= \arccos \left(-\frac{\delta q \nu + \alpha \rho}{\alpha_{\min}} \right). \end{aligned} \quad (\text{D.5})$$

Here,

$$I = \begin{cases} \frac{2}{\sqrt{1 - L^2}} \left[\arctan \left(\frac{\mu_0 + L}{\sqrt{1 - L^2}} \right) - \arctan \left(\frac{L}{\sqrt{1 - L^2}} \right) \right], & \text{for } L < 1, \\ \frac{1}{\sqrt{L^2 - 1}} \ln \frac{\mu_0(L + \sqrt{L^2 - 1}) + 1}{\mu_0(L - \sqrt{L^2 - 1}) + 1}, & \text{for } L > 1, \end{cases} \quad (\text{D.6})$$

with

$$\mu_0 = \frac{\delta q \nu \rho + \alpha - \alpha_{\min}}{\delta q \nu \sqrt{1 - \rho^2}}, \quad L = \frac{\alpha_{\min}(1 - \beta)}{K^{1-\beta} \nu \sqrt{1 - \rho^2}}. \quad (\text{D.7})$$

The forward price of an option is

$$\begin{aligned} P(K, f, \tau; \{\tilde{\alpha}, \tilde{\beta}, \tilde{\nu}\}) &= (f - K)^+ + \frac{2}{\pi} \sqrt{fK} \left\{ \int_{s_-}^{s_+} \frac{\sin(\eta \phi(s))}{\sinh s} G(\tau \nu^2, s) ds \right. \\ &\quad \left. + \sin(\eta \pi) \int_{s_+}^{\infty} \frac{e^{-\eta \psi(s)}}{\sinh s} G(\tau \nu^2, s) ds \right\}, \end{aligned} \quad (\text{D.8})$$

where the integration limits are defined as

$$s_- = \operatorname{arcsinh} \left(\frac{\nu |q - q_0|}{\alpha} \right), \quad s_+ = \operatorname{arcsinh} \left(\frac{\nu |q + q_0|}{\alpha} \right), \quad (\text{D.9})$$

with q from Equation (D.3) and

$$q_0 = \frac{f^{1-\beta}}{1 - \beta}. \quad (\text{D.10})$$

Furthermore,

$$\begin{aligned}\phi(s) &= 2 \arctan \sqrt{\frac{\sinh^2(s) - \sinh^2(s_-)}{\sinh^2(s_+) - \sinh^2(s)}}, \\ \psi(s) &= 2 \operatorname{artanh} \sqrt{\frac{\sinh^2(s) - \sinh^2(s_+)}{\sinh^2(s) - \sinh^2(s_-)}},\end{aligned}\tag{D.11}$$

and

$$\eta = \left| \frac{1}{2(\beta - 1)} \right|.\tag{D.12}$$

Finally, the kernel function $G(\tau, s)$ is given by

$$G(\tau, s) = 2\sqrt{2} \frac{e^{-\frac{\tau}{8}}}{\tau\sqrt{2\pi\tau}} \int_s^\infty u e^{-\frac{u^2}{2\tau}} \sqrt{\cosh(u) - \cosh(s)} du.\tag{D.13}$$

An approximation of the kernel function is provided in Antonov et al. (2013), Equation (11), which reduces Equation (D.8) to a set of single integrals:

$$G(\tau, s) \simeq \sqrt{\frac{\sinh(s)}{s}} e^{-\frac{s^2}{2\tau} - \frac{\tau}{8}} (R(\tau, s) + \delta R(\tau, s)),\tag{D.14}$$

where

$$\begin{aligned}R(\tau, s) &= 1 + \frac{3\tau g(s)}{8s^2} - \frac{5\tau^2(-8s^2 + 3g^2(s) + 24g(s))}{128s^4} + \frac{35\tau^3(-40s^2 + 3g^3(s) + 24g^2(s) + 120g(s))}{1024s^6}, \\ g(s) &= s \coth(s) - 1, \\ \delta R(\tau, s) &= e^{\frac{\tau}{8}} - \frac{3072 + 384\tau + 24\tau^2 + \tau^3}{3072}.\end{aligned}\tag{D.15}$$

For the purposes of this thesis, this approximation is used exclusively, as it offers a substantial gain in computational efficiency over the two-dimensional integrals. Moreover, as shown in the numerical experiments of Antonov et al. (2013), the approximation deviates from the two-dimensional integral solution by at most 0.3 basis points, showing that the loss in accuracy is negligible (p. 5).

List of Resources

Aid / Tool	Intended use
ChatGPT	Rewordings, aid in Python code writing and debugging, $\text{\LaTeX}$ layout
Grammarly	Grammatical checks
Overleaf	Writing and formatting of thesis
Python	Running simulations, training neural network, implementing approximations, creating visualizations

Declaration of Authorship

“I hereby declare:

- that I have written this thesis independently,
- that I have written the thesis using only the aids specified in the index;
- that all parts of the thesis produced with the help of aids have been declared;
- that I have handled both input and output responsibly when using AI. I confirm that I have therefore only read in public data or data released with consent and that I have checked, declared and comprehensibly referenced all results and/or other forms of AI assistance in the required form and that I am aware that I am responsible if incorrect content, violations of data protection law, copyright law or scientific misconduct (e.g. plagiarism) have also occurred unintentionally;
- that I have mentioned all sources used and cited them correctly according to established academic citation rules;
- that I have acquired all immaterial rights to any materials I may have used, such as images or graphics, or that these materials were created by me;
- that the topic, the thesis or parts of it have not already been the object of any work or examination of another course, unless this has been expressly agreed with the faculty member in advance and is stated as such in the thesis;
- that I am aware of the legal provisions regarding the publication and dissemination of parts or the entire thesis and that I comply with them accordingly;
- that I am aware that my thesis can be electronically checked for plagiarism and for third-party authorship of human or technical origin and that I hereby grant the University of St. Gallen the copyright according to the Examination Regulations as far as it is necessary for the administrative actions;
- that I am aware that the University will prosecute a violation of this Declaration of Authorship and that disciplinary as well as criminal consequences may result, which may lead to expulsion from the University or to the withdrawal of my title.”

By submitting this thesis, I confirm through my conclusive action that I am submitting the Declaration of Authorship, that I have read and understood it, and that it is true.



16.11.2025, Vincent Klaer